Inside Djex and Leant

A Maintainer's Walkthrough of the Codebases

Architecture, invariants, proof search, Lean integration,
and practical extension paths

Djex snapshot	`f50b5eac86726c7530d95e48c27101b7c0fa25c9`
Leant snapshot	`425131c6338d51f7b16189fdc0610da82e9cb641`
Relationship	Leant's `lib/Djex` submodule is pinned to the same Djex revision.
Prepared	August 10, 2026

Prepared for Vladimir Reshetnikov

This is a source-centered walkthrough, not an API brochure. It follows the actual control flow, data ownership, validation boundaries, and maintenance seams of the pinned revisions.

Abstract

Djex and Leant form a layered type-directed synthesis system. Djex is a Haskell library and command-line environment that gives two historically distinct synthesis engines—Djinn and Exference—a common, checked vocabulary for names, kinds, types, declarations, queries, candidates, generated syntax, diagnostics, and progress. Leant is an interactive Lean shell that links against Djex, speaks to a separate Lean backend process, translates elaborated Lean goals into a deliberately restricted structural language, runs one or both synthesis engines, reconstructs Lean syntax from the generated terms, and asks Lean itself to accept or reject every proposal.

The difficult engineering is not any single search routine. It is the preservation of meaning across a chain of representations that have different type systems and different notions of evidence. A Lean expression may be dependent, universe-polymorphic, nominal, recursive, and rich in implicit arguments. The synthesis engines reason over a smaller non-dependent fragment. The bridge must therefore distinguish exact structure from opaque transport, positive evidence from negative evidence, semantic assignments from pretty-printing hints, and exhaustive search from bounded search. Most of the code exists to make those distinctions explicit and enforceable.

This document walks through that code from the outside in. It explains the build graph; the opaque authority types in Djex; Djinn's proof-producing LJT pipeline; Exference's priority-ordered typed-hole search and independent checker; the unified Djex facade and frontends; Leant's backend protocol, transactional session reconstruction, snapshots, generated `it` bindings, goal-fragment serializer, provider discovery, staged search policy, candidate merger, renderer, and final Lean verification; and the test suites that lock those contracts down. The last part gives end-to-end traces and change recipes intended for someone modifying or reimplementing internals rather than merely calling the public API.

Contents

How to read this document

The walkthrough is organized around ownership and control flow rather than alphabetically by module. Each chapter contains source trails with commit-pinned links. Line numbers are intentionally omitted: they become misleading as soon as a file changes, while symbols and file-level responsibilities usually remain meaningful across refactorings.

A reader learning the system for the first time should read Chapters 1, 3, 5, 6, 9, and 15 in order. A maintainer debugging a failed synthesis should jump to the diagnostic funnels in Chapters 16 and 20. A contributor adding new type structure should start with the change recipes in Appendix C, then read the fragment and renderer chapters carefully.

The notation *authority type* means a type whose constructor is hidden and whose values can only be obtained through a validating smart constructor. This pattern is central to Djex. The notation *proposal* means a term produced by search but not yet accepted by the final authority. In Djex, a proposal may first be checked by a backend-specific checker; in Leant, it must ultimately be elaborated by Lean.

Post-snapshot addendum: typed behavioral assessment

The remainder of this walkthrough describes the pinned August 10 snapshots. The active development branch has since added a deliberately narrow typed-candidate and behavioral-assessment foundation. This addendum records that newer layer without rewriting the historical chapters or making their commit-pinned links point at code which did not exist at those revisions. Paths in this addendum are therefore printed as unlinked current-branch file paths rather than through the older commit-pinned `dpath`/`lpath` macros.

The headline is intentionally modest: the repositories now have a checked insertion seam for one finite-list-spine Length model and a scoped Z3 execution boundary, but Leant does not enable that path in `src/Main.hs`. The ordinary command and REPL behavior is unchanged. There is no claim here of general behavioral constraints, general program equivalence, a Lean-to-SMT semantics, solver-certified proofs, or completed user-facing Z3 integration.

The current authority chain

The newer path preserves one association across several independently checked boundaries. Reading it as a sequence is the easiest way to avoid giving a later stage authority which belongs to an earlier one.

1. **Exference retains a typed occurrence.** The typed Exference runners return an opaque `TypedCandidate`: the historical compatibility candidate remains paired with either its exact bounded `TermGraph` or an explicit engine-owned absence reason. When present, the graph's erasure was checked against that compatibility candidate. `synthesis/src/Language/Haskell/Synthesis/TypedGenerated.hs` seals graph structure and typing relationships, while `synthesis/src/Language/Haskell/Synthesis/TypedCandidate.hs` prevents callers from minting a forged checked association between that graph and another compatibility candidate. Both graph parameters—the neutral type and local-identity domains—are nominal. The sealed data constructor is private and has no derived `Generic` reconstruction path. Public callers can construct a raw `TermGraphSource`, but can obtain a sealed graph from it only through the bounded validating `sealTermGraph` entrance.

   Both engines erase this richer view through the same `typedQueryResultCompatibility` function for one already-checked `QueryResult`. It maps only the opaque candidate payload and therefore preserves the existing evidence, progress, metadata, candidate cardinality, and ordering, while leaving graph availability unobserved and candidate tails lazy. Djinn lifts that per-result projection over its singular error channel; Exference maps it over its lazy outer result trace. The different outer cardinalities remain engine-owned rather than being hidden behind another shared wrapper.

   Leant's `src/Leant/Synth/Engine.hs` retains that candidate in a `TypedCandidateSemanticSidecar` together with the exact Exference inventory/session, prepared translation origin, source-name table, converted source goal, session policy, request, provider assignments, and a lazy structural graph fingerprint attempt. Candidate grouping and renderer filtering preserve a group sidecar only while the group still denotes the same engine candidate. During combined-engine exact-text deduplication, an earlier compatibility-only spelling may instead retain a private variant-scoped witness to the first bounded later Exference occurrence with identical bytes. That witness keeps the Exference sidecar and source renderer ordinal separate from the display group's route, display ordinal, ordering, sidecar, and sibling variants. A wrapper such as the classical prove-mode transformation explicitly drops both direct and recovered authority.

2. **Lean callback acceptance is recorded, not inflated.** A rendered group carries its original renderer ordinal and semantic origin into `src/Leant/Synth/Verification.hs`. The bounded verifier tries variants lazily and mints an opaque nominal `Verified candidate` only when its callback returns `VariantAccepted`. In normal execution that callback asks the Lean backend to elaborate the generated verification program and rejects transport failures, fatal responses, diagnostics, and admitted `sorry` results. Nevertheless, the `Verified` type promises only callback acceptance at this boundary. It is not a Lean kernel proof object, behavioral evidence, or a solver certificate.

3. **The Length handoff rechecks correspondence.** `prepareCheckedLengthHandoff` in `src/Leant/Synth/Engine.hs` accepts the exact `Verified DetailedVerificationVariant`, not detached text. It fails closed unless that exact spelling has a direct or recovered typed Exference origin; engine and fitting fragments agree; no premise or request context retargets the problem; the search, request, and converted source goals still agree; the retained graph has not been lost; the origin's original renderer ordinal is zero; and that graph has exactly one direct Lean rendering equal to the callback-accepted text. A recovered origin therefore authorizes assessment of the identical accepted spelling, not the Djinn group or another textual variant.

   Only after those checks does the handoff resolve the declared unary spine and named providers through retained translation provenance. Djex then atomically seals the exact inventory, spine model, provider summaries, caller-supplied contract, typed candidate, and complete behavioral-problem identity. Raw text, candidate equality, private-name spelling conventions, and a graph recovered from a rendered term are not association mechanisms.

   The opaque successful handoff retains only that sealed problem. The callback receipt and resolved family binding have already served their checks and are not copied into the transient result; the later ranking record separately retains the one exact receipt required for occurrence-preserving presentation.

4. **The contract is explicit and solver-neutral.** `src/Leant/Synth/Length/Contract.hs` contains passive source vocabulary: exact Lean names for a unary zero/step spine, pre- and postconditions, and optional provider laws. Leant does not infer that contract from the goal, a declaration name, or an implementation. For each provider law the caller supplies argument roles and a Length transfer expression; the exact closed provider scheme and private identity come from retained translation provenance. Successful sealing establishes consistency of the named objects with the checked handoff. It does *not* establish that a caller-asserted provider law is true of the Lean implementation.

5. **Djex seals one versioned Length problem.** The public modules below `synthesis/src/Language/Haskell/Synthesis/Semantic/Length` implement only `finite-list-spine-length/v1`. The domain interprets a declared unary zero/step spine by natural-number length, supports a bounded expression/formula language, opens the exact candidate graph against the checked target, and records the precondition, postcondition, counterexample condition, inventory, encoding, candidate, and problem fingerprints. This is not a semantics for arbitrary inductives, dependent values, effects, partiality, infinite structures, or arbitrary Haskell/Lean programs.

6. **Canonical queries precede execution.** `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs` can seal a bounded canonical QF_LIA query only from one checked Length problem. Query construction is pure and has its own size and numeral limits. `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs` separately validates an explicit executable path, optional SHA-256 expectation, solver timeout and resource limit, host deadline, response bounds, and artifact policy. Constructing either value performs no process IO and grants no solver authority.

7. **Live Z3 ownership is lexical and causal.** `synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs` lends an opaque session only through a rank-2 callback. One capability-probed worker serves at most 64 serial queries in that lexical scope; no process handle, marker, ordinal, transcript, raw decoded valuation, workspace, or reusable run identity escapes. Package-private protocol and session modules use the shared `SMTLibCausalAction` algebra in `synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Causal.hs`: an exact write obligation is completed and flushed before its receiver may consume causally attributable bytes. Positional barriers, per-query ordinals, bounded framing, whitespace-only write boundaries, deadlines, poisoning, and cleanup reject stale or misassociated output rather than scanning forward for a convenient response.

   The executable digest is a bounded pre-spawn file observation, not attestation of the executed image or its loader and libraries. The live facade exposes only sanitized status, association, evidence, and closed failure classes. If invoked, the child solver is a fourth computational domain whose session authority is lent only for this dynamic scope; `src/Main.hs` does not currently invoke it, so the default three-domain runtime described in Chapter 1

remains accurate. A worker belongs to one attempted ranking batch. Cleanup is owned and attempted when the scope exits, incomplete cleanup is reported, and no worker becomes long-lived interactive session state.

8. **Status and evidence stay separate.** `sat`, `unsat`, and `unknown` are all retained only as `HeuristicRankingOnly` solver observations. In particular, `unsat` is relative to the bounded encoding and is neither a solver certificate nor pruning authority; status-only `sat` has no model evidence. Under the input-values policy, a satisfiable response can yield evidence only after exact symbol decoding and an independent concrete natural-number replay against the checked Length problem. Djex's `replayLengthSMTLibLiveQueryObservation` checks the returned query fingerprint before inspecting optional evidence, then replays that opaque evidence once more against the exact behavioral problem retained by the sealed query. Leant accepts a `ValidatedLengthCounterexample` only through that gate; the heavier handoff is transient after query sealing.

   Even that receipt is deliberately limited. It says that one bounded finite-spine natural assignment violates the sealed postcondition under the sealed precondition and model. If the candidate used named provider laws, the receipt explicitly remains conditional on those assumptions. It is not a concrete Lean value, a source-level execution, a solver-soundness theorem, or a kernel proof.

9. **Assessment may reorder but never prune.** The package-private implementation in `src/Leant/Synth/Length/Ranking/Internal.hs` first productively admits at most 64 candidates and completes all pure candidate/query preparation before opening a worker. In the successful order, `Unassessed` candidates and candidates carrying only heuristic statuses retain their relative order; candidates with independently replayed counterexamples follow, also stably. Every original callback receipt remains present. Any returned session, query, fingerprint-association, or evidence-replay failure discards partial live assessments and restores the original order with all assessments `Unassessed`.

   Pure candidate-local preparation refusals use ten fixed payload-free classes: unsupported route, typed authority unavailable, candidate association rejected, rendering association rejected, spine binding unavailable, provider binding unavailable, session rejected, contract rejected, candidate semantics rejected, and query construction rejected. Their fixed codes reveal only the phase. The classifier does not evaluate or copy renderer text, source names, types, graph identities, or nested Djex errors from a raw refusal into that diagnostic; the exact verified receipt and semantic sidecar remain attached to the candidate for association. Such a pre-execution refusal survives atomic batch fallback; a candidate which reached execution gets no fabricated preparation reason. Refusal classes are diagnostics, not behavioral strength and not ranking input.

10. **A generative seal protects the final permutation.** `src/Leant/Synth/PostVerification.hs` introduces a fresh rank-2 epoch for the exact opaque `VerificationBatch`. An assessor may reorder only the supplied private occurrence handles. The seal productively bounds both lists and requires equal length, in-range indices, and no duplicate occurrence, so it cannot omit, manufacture, duplicate, or substitute an equal-valued candidate. Equal payloads remain distinct occurrences. The Length adapter in `src/Leant/Synth/Length/PostVerification.hs` carries each assessment with its batch-scoped handle through ranking and seals the handle permutation before erasing that association.

Current post-verification invariant

The only candidate batch eligible for sealed post-verification presentation and reordering through the Length adapter is the exact callback-accepted batch. The only current reason to move an occurrence is an independently replayed `finite-list-spine-length/v1` counterexample for that exact retained Exference graph and the canonical query sealed from its checked handoff. No solver status removes a candidate, and a complete bounded permutation is sealed before presentation. Association-free compatibility runners remain lower-level reports and do not themselves grant presentation authority.

Djinn uses the envelope without inventing graph authority

Djinn now exposes `DjinnTypedCandidate` and `DjinnTypedResult`, with typed counterparts of its ordinary, provider-candidate, assignment, and kinded-assignment runners in `djinn/src-core/Language/Haskell/Djex/Djinn.hs` and `djinn/src-core/Djinn/Core.hs`. The legacy runners lift the shared one-way `typedQueryResultCompatibility` projection over those singular typed paths; Exference uses the same per-result edge inside its lazy result trace. Final

candidate keys are allocated only after proof checking, deduplication, and any configured final ordering step, so future typed occurrence identities need not depend on discarded proof-plan ordinals.

This common result shape is not a license to synthesize a graph. Djinn's independently checked LJT evidence precedes source-name restoration, provider rewriting, instantiation-evidence lowering, visible type applications, pattern refinements, simplification, and final naming; structural formula expansion has also discarded source distinctions needed for a shared typing derivation. Every current Djinn typed candidate therefore reports the explicit absence `DjinnTermGraphSourceTypingContextUnavailable`. The absence is a capability result, not a search failure and not weaker logical evidence.

The reserved future graph type retains variable role. The literal right-hand side of `DjinnTermGraphTypeVariable` is `SharedType.Variable HSymbol`; `DjinnTermGraphType` then applies `SharedType.Type` to that alias. This differs from the historical role-erased compatibility type. Constructing it soundly will require retained source authority and a bounded bidirectional check of the final generated clause with exact compatibility erasure. Until such a graph exists, Djinn candidates are not eligible for the Length handoff, Z3 ranking, or any other graph-based behavioral contract. Code must not replace the explicit absence by guessed annotations reconstructed from final syntax.

Module map for the added foundation

The following paths are current-branch source locations, not links into the pinned walkthrough snapshots.

Djex path	Authority or responsibility
`synthesis/src/Language/Haskell/Synthesis/TypedGenerated.hs`	Bounded typed term graph, checked erasure, nominal type/local domains, and graph construction limits.
`synthesis/src/Language/Haskell/Synthesis/TypedGenerated/Fingerprint.hs`	Allocation-insensitive structural fingerprint for a sealed graph.
`synthesis/src/Language/Haskell/Synthesis/TypedCandidate.hs`	Public opaque typed-candidate facade and shared evidence-preserving compatibility projection for one checked query result.
`synthesis/internal/Language/Haskell/Synthesis/Internal/TypedCandidate.hs`	Private representation, nominal roles, checked construction seam, lazy selectors, and canonical implementation of the per-result projection.
`exference/src-core/Language/Haskell/Djex/Exference.hs`	Public Exference typed runners and checked-candidate projection.
`exference/src-core/Language/Haskell/Exference/Core/Internal/ExpressionCheck.hs`	Private typed reconstruction which admits completed Exference expressions and checks graph erasure before graph projection.
`djinn/src-core/Language/Haskell/Djex/Djinn.hs`	Public Djinn typed runners and compatibility projections.
`djinn/src-core/Djinn/Core.hs`	Proof-checked result associations, final ranking/deduplication, and explicit typed graph absence.
`djinn/src-internal/Djinn/Internal/CheckedCandidate.hs`	Private validated-candidate association and post-ranking keyed projection seam.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Problem.hs`	Generic behavioral-problem identity, heuristic observation association, and replay-gated opaque evidence.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length.hs`	Versioned finite-list-spine Length domain, limits, expressions, formulas, spine and provider vocabulary.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Problem.hs`	Atomic checked session, provider inventory, candidate semantics, and complete Length problem.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/Evaluate.hs`	Bounded concrete natural-number evaluator and the only constructor path for validated Length counterexamples.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib.hs`	Canonical bounded QF_LIA query, query fingerprint, and model-binding validation.
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Execution.hs`	Pure explicit execution policy and response/resource admission.

Djex path	Authority or responsibility
`synthesis/src/Language/Haskell/Synthesis/Semantic/Length/SMTLib/Live.hs`	Sanitized rank-2 live-session facade, replayed-evidence observation, and exact query-first evidence-consumption gate.
`synthesis/internal/Language/Haskell/Synthesis/Internal/SMTLib/Causal.hs`	Shared nominal write/await/complete action algebra for pure SMT-LIB machines.
`synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Protocol.hs`	Pure positional Length query protocol and causal write boundaries.
`synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session/Driver.hs`	One process-owning causal driver shared by readiness and query machines.
`synthesis/internal/Language/Haskell/Synthesis/Internal/Semantic/Length/SMTLib/Session.hs`	Scoped lease, serial ordinals, marker ownership, query execution, poisoning, replay, and cleanup.
`src/Language/Haskell/Djex.hs`	Curated public re-export of the typed-candidate, semantic Length, SMT-LIB, and scoped live surfaces.

Leant path	Authority or responsibility
`src/Leant/Synth/Engine.hs`	Exference run authority, typed semantic sidecar, renderer/verification provenance, and checked Length handoff.
`src/Leant/Synth/Verification.hs`	Lazy bounded callback verification and opaque nominal acceptance receipts.
`src/Leant/Synth/Length/Contract.hs`	Solver-neutral caller-supplied spine, condition, and provider-law vocabulary.
`src/Leant/Synth/Length/Adapter.hs`	Pure conversion from one checked handoff to one canonical Length query.
`src/Leant/Synth/Length/Configuration.hs`	Opaque explicit execution/replay policy; contract remains request-owned.
`src/Leant/Synth/Length/Configuration/File.hs`	Closed, bounded version-1 JSON decoder returning a disabled configuration value.
`src/Leant/Synth/Length/Configuration/File/Acquire.hs`	Explicit-path, bounded, sanitized POSIX descriptor acquisition; no discovery or activation.
`src/Leant/Json/Bounded.hs`	Shared strict bounded JSON syntax, scalar decoding, and closed-object validation used by the configuration boundary.
`src/Leant/Synth/Length/Ranking.hs`	Association-free ranking facade: complete reports, sanitized failures and refusals, and no associated-plan projector.
`src/Leant/Synth/Length/Ranking/Internal.hs`	Package-private transient handoff preparation, verified-receipt/query state, scoped serial execution, query-owned evidence replay, handle-preserving no-prune ordering, and atomic fallback.
`src/Leant/Synth/PostVerification.hs`	Domain-neutral generative occurrence handles and bounded exact-permutation seal.
`src/Leant/Synth/Length/PostVerification.hs`	Safe Length-specific facade exposing only configured or policy-plus-contract sealed assessment.
`src/Leant/Synth/Length/PostVerification/Internal.hs`	Package-private rank-2 adapter which seals occurrence handles before projecting the ranking report.
`src/Main.hs`	Current identity-disabled integration point: callback verification followed by `skipPostVerificationAssessment`.

What is deliberately not wired

`src/Main.hs` currently calls `skipPostVerificationAssessment` immediately after `synthVerify`. That function is a non-strict projection of the callback receipts in their existing order. It performs no IO, does not read or discover

a configuration file, does not select a contract, does not open Z3, and does not claim that permutation validation occurred. The displayed `it` bindings therefore retain the historical callback order.

The configuration modules establish an explicit future policy boundary without silently activating it. The pure version-1 decoder returns an opaque disabled value; the acquisition module requires a caller-supplied absolute path and timeout and has no project, environment, home-directory, or executable-relative discovery convention. Activation, including the choice between a pinned executable and an explicitly permitted unpinned executable, remains a separate caller decision. A future command or configuration owner must also choose the request's Length contract and define how acquisition or live-ranking failure affects presentation.

If that path is enabled later, it should replace only the named post-verification identity seam. It must continue to use the exact `VerificationBatch`, a sealed explicit policy, and one request-owned contract; open a fresh lexical worker scope; preserve sanitized diagnostics and atomic fallback; and present candidates only after the occurrence permutation seal succeeds. Broader behavioral domains should be added as new checked domain adapters with their own semantics and replay authorities, not by treating `finite-list-spine-length/v1` as a generic constraint language.

Do not read readiness as completion

The presence of typed envelopes, canonical SMT-LIB, a live Z3 facade, bounded configuration acquisition, and a post-verification seal means the trust boundaries can now be composed. It does not mean Leant presently starts a solver, that Djinn has a typed graph, that provider laws were proved, that `unsat` proves correctness, or that the system supports arbitrary behavioral specifications.

Part I

The joint system

Chapter 1

A precise mental model

1.1 What runs where

There are three computational domains:

1. The **Leant process** is a Haskell executable. It owns the interactive command loop, user-visible state, Djex sessions, synthesis policy, renderer, provider cache, transcript, history, and snapshots.
2. The **Lean backend process** is started through `lake env`. It elaborates declarations and commands, owns Lean environment identifiers, serializes elaborated goals, prints declarations, checks generated candidates, and creates snapshot artifacts.
3. The **Djex library code** is linked into Leant. It does not communicate with Lean directly. It receives a checked neutral environment and query, runs Djinn or Exference in-process, and returns neutral generated expressions plus evidence and progress.

This distinction matters. Leant is not a text-only wrapper around a Djex command-line program, and Djex is not a plugin loaded into the Lean kernel. Leant is the semantic bridge. It uses Lean as the source and final authority, while using Djex as an in-process synthesis service.

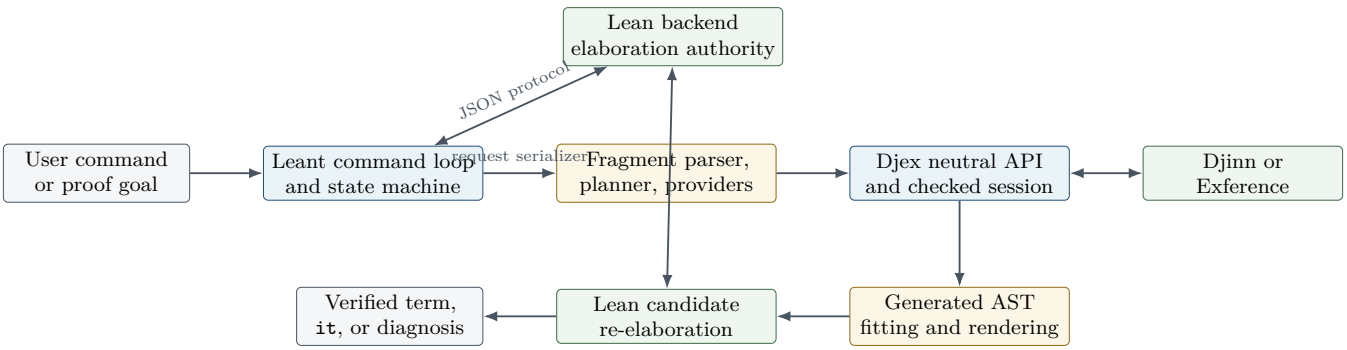

Figure 1.1: The runtime boundary. Lean and Djex are both authorities, but for different propositions. Djex validates and searches its own fragment; Lean decides whether the reconstructed source term actually inhabits the original Lean goal.

1.2 The representation ladder

A single successful `:synth` crosses at least six representations:

Representation	Owner	Purpose and information content
Elaborated Lean expression	Lean backend	Full dependent expression, universes, implicit binders, reducible aliases, nominal constants, constructors, instances, and local context. This is the source truth.
Serialized fragment	Lean serializer in Leant's payload	A bounded S-expression describing only the structure Leant knows how to transport or analyze: arrows, products, sums, quantifiers, selected inductives, exact contexts, applications, atoms, and explicit incompleteness markers.
`Frag` value	Leant Haskell code	Parsed and analyzed bridge representation. It carries enough metadata to decide what search may inspect, what must remain opaque, how to render a candidate, and whether a negative result could be sound.
Djex neutral type and declarations	Shared Djex synthesis layer	Backend-independent Haskell type language, environment, class information, exact provider-instantiation evidence, query options, and generated-expression vocabulary.
Backend internal state	Djinn or Exference	Djinn uses formulas, sequents, proof terms, and structural plans. Exference uses typed holes, scopes, substitutions, a priority queue, constraints, and usage/rating metadata.
Shared generated AST	Djex	A checked neutral term language containing locals, globals, application, visible type application, tuples, lambdas, lets, cases, and patterns. It deliberately does not pretend to be Lean syntax.
Rendered Lean text	Leant	One or more source spellings after premise stripping, constructor restoration, binder uniquification, eta fitting, explicit-argument insertion, provider substitution, and idiomatic/fully explicit rendering.
Verified Lean term	Lean backend	The first spelling that elaborates against the exact original goal without an error, fatal response, or admitted proof.

A recurring maintenance mistake is to collapse two adjacent rungs. For example, provider-instantiation evidence is semantic data used to justify a backend instantiation; provider-rendering metadata is source data used to reconstruct the right Lean spelling. They often describe the same occurrence, but they are not interchangeable.

Invariant

No stage is allowed to infer more certainty than the previous stage supplied. Opaque source structure may be transported, but it may not silently become decomposable. A bounded search may produce candidates, but exhaustion under a bound may not silently become a refutation. A pretty-printed type may guide diagnostics, but it may not silently become exact type evidence.

1.3 Two independent axes: evidence and progress

Both codebases take unusual care to keep **what the search established** separate from **why it stopped**. These are independent axes.

A search can finish and find candidates. It can finish and prove that the translated goal is uninhabited. It can finish without either kind of evidence because the backend is heuristic. It can be truncated after already finding candidates. It can also be truncated without finding a candidate, which is the weakest possible result. Djex therefore models progress with values such as `Finished` and `Truncated reason`, while modeling evidence separately. Leant then adds another filter: even a genuine Djex refutation may be unusable if the Lean-to-Djex projection hid relevant structure.

The practical consequence is a verdict lattice rather than a Boolean:

Search observation	Safe user-level interpretation	Unsafe shortcut
Candidate and successful Lean check	The original Lean goal is inhabited by the checked term.	None; this is positive evidence at the final authority.
Djinn refutation, complete projection, no budget	The projected propositional goal is uninhabited; Leant may report a sound negative result.	Reporting it without checking fragment completeness.
Djinn no proof under budget	No candidate found within the budget.	Calling the goal uninhabited.
Exference queue exhausted	No candidate found by that heuristic search.	Treating queue exhaustion as a proof of impossibility.
Rendered candidate rejected by Lean	That spelling does not elaborate against the exact goal.	Concluding that the underlying semantic candidate is invalid; another spelling may work.

1.4 The trust chain

There are several checkers, and they prove different things:

1. Djex smart constructors prove local representation invariants: names are valid, declarations are coherent, types are finite where required, queries are scoped, and exact assignment vectors satisfy widths and kinds.
2. Djinn's proof checker proves that an LJT proof term derives the translated formula under the compiler's proof environment.
3. Exference's expression checker reconstructs the candidate independently from search, using only the shared unification kernel and explicit provenance where necessary.
4. Djex generated-syntax validation proves that the neutral term is structurally scoped and renderable in its own vocabulary.
5. Lean elaboration proves the proposition users actually care about: the rendered term has the original Lean type in the original Lean environment.

This is defense in depth, not redundancy. The internal checkers catch engine defects early and make Djex usable outside Lean. The final Lean check closes the semantic gap introduced by projection and source reconstruction.

Source trail

Start with the Djex architecture narrative in `docs/architecture.md`; the neutral facade in `src/Language/Haskell/Djex.hs`; Leant's process boundary in `src/Leant/Backend.hs`; and the top-level orchestration in `src/Main.hs`. The rest of this document decomposes those paths into maintainable subsystems.

Chapter 2

One synthesis request, end to end

This chapter gives a compact trace before the detailed subsystem chapters. Suppose the user asks Leant to synthesize a term for a goal morally equivalent to

```
forall {a b : Sort u}, (a -> b) -> a -> b
```

The exact expression may contain universe levels, binder annotations, and local declarations. The bridge follows these stages.

2.1 Stage 1: establish the exact Lean goal

Leant determines the goal source. In ordinary mode this can be an explicit type or the type surrounding the most recent `sorry`. In prove mode it is the current target. Leant ensures that the derived synthesis environment corresponds to the current interactive environment; if declarations or imports changed, the synthesis base is rebuilt before any provider query is trusted.

Leant asks the Lean backend to execute a serializer payload. The payload runs after elaboration, so it examines *expressions*, not surface text. It emits a bounded S-expression. The serializer records explicit versus implicit forall binders, instance/context information, the structural connectives it recognizes, and metadata for selected inductive occurrences. When it cannot safely descend, it emits an opaque or depth marker instead of guessing.

2.2 Stage 2: parse and classify the fragment

`Leant.Synth.Fragment` parses the S-expression into `Frag`. It then derives several views:

- the leading binder spine, needed to align generated lambda binders with Lean's explicit, implicit, and instance slots;
- recursive-family keys, used to avoid importing irrelevant recursive constructor schemas;
- unsafe atoms and unsupported context markers, which may poison negative evidence but do not necessarily prevent positive synthesis;
- provider-query roots and result heads;
- a completeness summary for recursive and nominal projections;
- optional library premises serialized in the same batch.

For the example, the core fragment is a chain of foralls and arrows. Bound sort variables remain connected across the domains and codomain; they are not replaced with unrelated pretty-print names.

2.3 Stage 3: choose search lanes

Leant does not immediately dump every visible Lean declaration into one enormous search. It tries a staged policy:

1. a structural, engine-only run;
2. when enabled, a separately budgeted curated-library run;
3. when the goal permits it, live-provider runs with gradually widening provider prefixes;
4. optionally, a classical fallback for the precisely characterized quantifier-free fragment.

The first stage is enough for the example because the target follows from its arguments alone. No provider discovery is needed.

2.4 Stage 4: construct a checked Djex query

`Leant.Synth.Engine` translates `Frag` into the shared Djex `Type`, creates any synthetic declarations and premises, constructs exact provider-instantiation assignments if providers are active, and builds a `QueryRequest`. The target name is a private synthetic definition such as `leantSynth`; the goal is the translated type; contexts are checked; backend options carry candidate, step, queue, and optional choice-point limits.

The backend adapter converts that public request into an opaque checked request associated with a validated session. This is where session-independent and session-dependent preflight occurs. A malformed request is rejected before search.

2.5 Stage 5A: Djinn path

Djinn compiles the translated type into an intuitionistic formula. For the example, after opening the positive universal prefix, the formula is essentially

$$(a \to b) \to a \to b.$$

The LJT search introduces the two antecedents and applies the first to the second. It returns a proof term. The proof checker independently validates the derivation. The lowering pass converts the checked proof to the shared generated AST, approximately

```
Lambda [Bind "f", Bind "x"]
  (Apply (Local "f") (Local "x"))
```

The candidate stream carries positive evidence. Search progress records whether more candidates were omitted or a bound was reached.

2.6 Stage 5B: Exference path

Exference creates a root typed hole at the goal. Opening the forall prefix extends the rigid type scope. Introducing the arrows extends the term scope with `f` and `x`. To fill the result hole of type `b`, the search considers in-scope values and globals. Selecting `f : a -> b` creates an argument hole of type `a`; selecting `x : a` solves it. The resulting expression is simplified and then reconstructed by the independent expression checker.

Exference may rank several candidates and may stop due to step, depth, identifier, or queue limits. Even when its queue empties, it produces no proof of uninhabitability.

2.7 Stage 6: fit and render

Leant receives the neutral expression. The renderer strips synthetic premise binders, restores exact Lean global and constructor names, normalizes patterns into forms Lean accepts, uniquifies every binder, aligns lambda binders with the goal's explicit/implicit spine, inserts visible type placeholders where necessary, eta-expands selected forms, substitutes provider occurrences, and emits bounded source variants.

For the example, a likely first spelling is:

```
fun f x => f x
```

A more explicit variant may also be retained when ambiguity exists. Variants are grouped because they represent one semantic candidate.

2.8 Stage 7: ask Lean

Leant sends a verification payload to the Lean backend. The payload checks each candidate against the exact original goal. A successful spelling is accepted; errors, fatal responses, and admitted terms are rejected. Verified candidates are displayed. Outside prove mode, Leant binds them as generated definitions in reverse quality order so the best surviving result becomes the newest bare `it`. In prove mode, it records proof splices that apply the candidate to the current hypotheses.

Invariant

A candidate only becomes a Leant result after the exact Lean backend accepts it. Djinn's proof and Exference's checker establish strong internal claims, but neither is allowed to substitute for final elaboration across the language boundary.

Part II

Djex

Chapter 3

Repository, package, and module map

3.1 The package is one component with several source roots

The top-level `djex.cabal` describes one package that deliberately preserves several historical trees while exposing a converged library. The important source roots are:

Root	Responsibility
`synthesis/src`	Shared synthesis vocabulary and validation: names, kinds, types, type atoms, constraints, declarations, environments, inventories, kind inference, queries, search progress, candidates, generated terms, diagnostics, selection, and rendering helpers.
`djinn/src-core`	Djinn's checked adapter, session/request internals, type-to-formula compiler, LJT proof search, proof checker, instantiation machinery, and proof-to-generated lowering.
`djinn/src-frontend`	Historical Djinn-compatible API and REPL facade. It is important for compatibility, but it should not own shared invariants.
`exference/src-core`	Exference's checked adapter and its engine: unification, class constraints, typed expression, typed holes, scopes, queue scheduler, scoring, rigid instantiation, independent expression checker, and diagnostics.
`exference/src-frontend`	Historical Exference source parser, environment loader, Haskell conversion layer, and CLI.
`src`	Unified Djex facade, unified command line and REPL, package/workspace handling, source translation, and the small public entry surface used by new clients.

The Cabal file also exposes historical `djinn` and `exference` executables alongside `djex`. This is not merely packaging clutter. It lets the project maintain compatibility tests while moving semantic ownership into shared modules.

3.2 The central architectural rule

The package converges infrastructure without pretending the engines are the same algorithm. Shared modules own contracts that both engines must honor. The adapters own conversion and backend policy. Djinn and Exference retain distinct search representations, ordering, completeness properties, and diagnostics.

This rule explains many otherwise surprising boundaries:

- The shared `Type` is rich enough for both engines, but neither backend is forced to inspect every constructor identically.
- The shared `Generated` AST is a target language, but each backend may reach it by a different checked route.

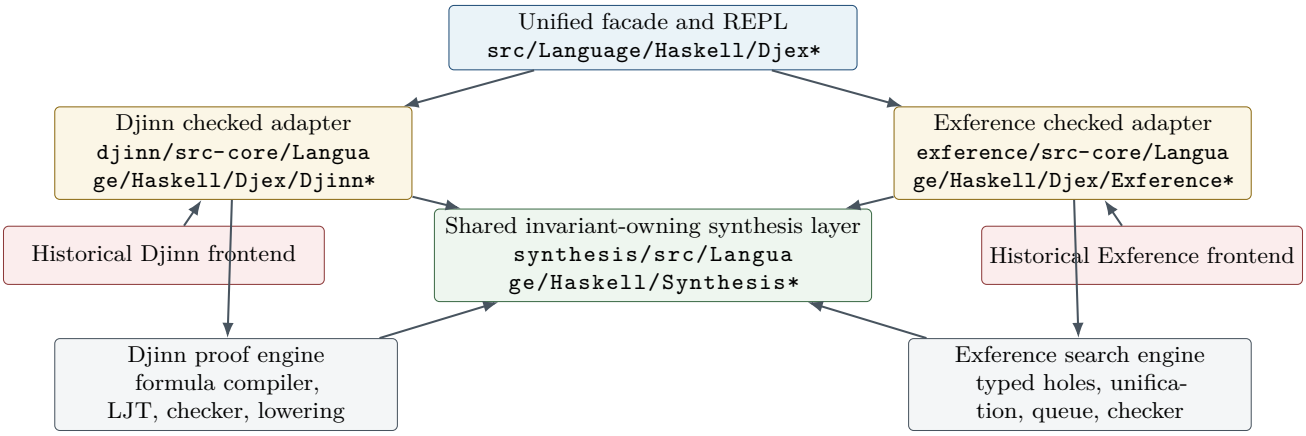

Figure 3.1: Djex's dependency intent. Shared semantic types sit below the adapters; backend-specific search remains sealed. The historical frontends point inward rather than defining new semantic authorities.

- Search progress is shared, but the meaning of queue exhaustion differs from proof-search exhaustion.
- Provider-instantiation assignments are shared evidence, but each backend validates and consumes them in its own request preparation.
- The facade can expose a capability matrix instead of lying that every backend has every property.

Why it is designed this way

A tempting unification would be one giant `SearchState` and one generic step function parameterized by callbacks. That would erase exactly the information that makes the engines valuable: Djinn's proof-theoretic completeness story and Exference's ranking, type-directed expansion, and source-oriented heuristics. Djex instead unifies the borders and leaves the centers different.

3.3 Recommended first reading order

For a new maintainer, the shortest path to a correct mental model is:

1. `synthesis/README.md` for the shared contract;
2. `synthesis/src/Language/Haskell/Synthesis/Type.hs`, then `synthesis/src/Language/Haskell/Synthesis/Environment.hs`;
3. `synthesis/src/Language/Haskell/Synthesis/Query.hs`, `synthesis/src/Language/Haskell/Synthesis/Search.hs`, and `synthesis/src/Language/Haskell/Synthesis/Generated.hs`;
4. `djinn/src-core/Language/Haskell/Djex/Djinn.hs` and `djinn/src-core/Djinn/Core.hs`;
5. `exference/src-core/Language/Haskell/Djex/Exference.hs` and `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`;
6. `src/Language/Haskell/Djex.hs` and the unified REPL modules.

Do not start by reading the two largest backend files linearly. First learn what their inputs and outputs are permitted to mean. Otherwise implementation details such as freshness supplies, proof-plan families, or queue pruning have no frame of reference.

Chapter 4

The shared synthesis foundation

4.1 Names are checked data, not strings

The shared name layer in synthesis/src/Language/Haskell/Synthesis/Name.hs distinguishes namespaces and syntactic roles. Values such as identifiers, type constructor names, data constructor names, class names, method names, and definition names are not freely interchangeable. Smart constructors validate spellings and reserved forms; rendering can therefore rely on the distinction without reparsing arbitrary text.

This is a small example of the project's general style: validate once, hide the constructor, and pass an authority value downward. A backend should not have to ask repeatedly whether the query target is a legal definition name.

The layer also separates semantic identity from qualification and source spelling. That matters because a generated term may need a fully qualified source name even though environment lookup used a canonical key, and because a local binder must never be confused with a global declaration merely because the text matches.

4.2 Kinds and the neutral type language

The neutral type language is centered in synthesis/src/Language/Haskell/Synthesis/Type.hs. At a high level it contains:

```
data Variable id
  = FlexibleVariable id
  | RigidVariable id

data Type v
  = TypeVariable v
  | TypeConstructor Name
  | TypeApplication (Type v) (Type v)
  | FunctionType (Type v) (Type v)
  | TupleType Boxity [Type v]
  | ForallType [v] [Constraint (Type v)] (Type v)
```

The precise exported surface is richer, but this sketch captures the semantic split. Flexible variables are unification variables in Exference and instantiation variables in shared validation. Rigid variables are constants scoped by a binder or exact assignment. A forall stores its binders and class constraints explicitly. Tuples retain boxity and arity rather than being flattened to anonymous constructor applications at the public boundary.

4.2.1 Canonical forms without erasing source distinctions

The module provides canonical function and tuple views, strict application and function spines, constructor-application views for higher-kinded unification, free-variable and binder analysis, global binder renaming, and capture-avoiding simultaneous substitution.

The important qualification is *without erasing source distinctions*. Normalization may turn a saturated representation into a canonical form for comparison, but it must not dissolve a nested polytype into ordinary first-order nodes or forget that a nominal constructor came from a particular source declaration.

For example, higher-kinded unification wants to observe

$$F\ a\ b$$

as a head plus arguments, and it wants functions and tuples to participate in the same application machinery. That is the purpose of a constructor-application view. It is not permission to rewrite every type permanently into an untyped list of symbols.

4.2.2 Finite observation boundaries

Many public values are Haskell lists, and clients can accidentally supply cyclic or infinite lists. Constructors that must establish a finite invariant do not blindly call `length`. They observe only a documented maximum width plus a sentinel element, then reject oversize or nonterminating shapes at a bounded frontier where possible. The same principle appears in provider assignment vectors, class arities, kind trees, generated hints, and search prefixes.

Invariant

Strict authorities never retain a value whose validity depends on consuming an unbounded lazy tail. Laziness is reserved for search result streams and other places where incremental observation is itself the API.

4.3 Type atoms: opaque, alpha-aware islands

`synthesis/src/Language/Haskell/Synthesis/TypeAtom.hs` solves a particularly difficult problem. A nested polymorphic type can be meaningful as a whole without being safe to decompose in a first-order unifier. Djex therefore represents such a region as a `TypeAtom`: an exact source tree paired with an alpha-normal lexical key and enough variable information for substitution and occurs checks.

Ordinary unification treats the atom as opaque. Equality remains alpha-aware. Free variables inside the atom still participate in capture avoidance and occurs checks. Thus Djex avoids two unsound extremes:

- treating the atom as a completely uninterpreted string, which would lose alpha-equivalence and variable provenance;
- opening the atom as if rank-N structure were ordinary first-order syntax, which would allow invalid decomposition and escaping binders.

Type atoms recur in both backends. Djinn may compile a nested negative forall to an opaque proposition symbol keyed by a type atom. Exference's unifier may bind an opaque polytype as one unit but cannot descend into it.

4.4 Constraints, classes, declarations, and synonyms

The shared declaration vocabulary spans `synthesis/src/Language/Haskell/Synthesis/Constraint.hs`, `synthesis/src/Language/Haskell/Synthesis/Class.hs`, `synthesis/src/Language/Haskell/Synthesis/Declaration.hs`, and `synthesis/src/Language/Haskell/Synthesis/TypeSynonym.hs`. It models value signatures, data/type declarations, classes, instances, methods, constraints, and type synonyms in source order.

Source order is significant. A declaration can only refer to authorities already established by the prefix unless a specific recursive form permits otherwise. Type-synonym expansion must terminate or report a cycle. Class validation must know arity, superclass prerequisites, method ownership, and instance-head shape. Those checks belong before either backend builds a session.

The shared kind inference module, `synthesis/src/Language/Haskell/Synthesis/KindInference.hs`, validates type applications and exact provider assignments across proper and higher kinds. This is why provider evidence carries kind information rather than a flat list of types.

4.5 Environment as an authority

synthesis/src/Language/Haskell/Synthesis/Environment.hs is one of the most important files in the repository. The public `Environment` is opaque. Internally it retains the source declarations and several strict indexes: types, values, constructors, classes, instances, instance heads, and occupied names. `mkEnvironment` validates the entire declaration sequence before constructing those indexes.

The indexes are duplicated views of one semantic state. An update must therefore be transactional: either the declaration and all relevant indexes advance together, or the old environment remains. Code that appends a declaration to the source list but forgets an instance-head index would create an internally contradictory authority.

A useful way to read the constructor is as a fold over declarations with a large invariant:

```
step :: CheckedPrefix -> Declaration -> Either Diagnostic CheckedPrefix
step prefix declaration = do
  validateNamespaces prefix declaration
  validateKinds      prefix declaration
  validateClasses    prefix declaration
  validateInstances  prefix declaration
  updateEveryIndex   prefix declaration
```

This is explanatory pseudocode, not a literal excerpt. The actual code has more specialized paths, but the transactional model is accurate.

4.5.1 Inventory and prepared inventory

synthesis/src/Language/Haskell/Synthesis/Inventory.hs derives a search-oriented inventory from the environment. A prepared inventory is another opaque authority: aliases are expanded where permitted, recursive components are classified, and lookup tables are normalized for backend consumption.

The project also distinguishes a transient *prepared-inventory expansion*. That view may expose alias-free structures and recursion metadata for a particular operation, but it is not retained as a long-lived authority. This prevents a temporary expansion policy from silently becoming the canonical meaning of the environment.

Why it is designed this way

The environment answers, "What declarations exist and are they coherent?" The inventory answers, "What search-facing facts can be derived from that environment?" A prepared expansion answers, "What does this particular type look like after this bounded expansion policy?" Keeping these questions in separate types prevents a backend convenience from mutating source semantics.

4.6 Queries and checked requests

The public query language in synthesis/src/Language/Haskell/Synthesis/Query.hs contains a target definition name, a goal type, explicit contexts, and backend options. Provenance and display metadata may accompany the query, but semantic equality does not accidentally depend on incidental diagnostics.

A `CachedQuery` is opaque. It represents the result of normalization and validation that can safely be reused. The backend adapters then add session-specific checks and wrap it in private `DjinnRequest` or `ExferenceRequest` types. This two-stage preparation has a practical payoff: malformed shape is rejected once, while environment-dependent facts are checked against the exact session that will execute the query.

Provider evidence is part of the request contract. The shared forms distinguish:

- a provider-instantiation candidate, describing a possible relation between a provider and a query occurrence;
- an assignment, choosing concrete arguments for that provider;
- a kinded assignment, retaining each argument's kind and exact representation.

The vectors are bounded. At this snapshot, a provider assignment supports at most six arguments, with finite caps on candidates and assignments. These limits are not merely performance knobs; they are part of the validation boundary that keeps exact evidence inspectable.

4.7 Search progress and evidence

synthesis/src/Language/Haskell/Synthesis/Search.hs defines backend-neutral progress. Truncation reasons include step and choice-point limits, candidate cutoffs, identifier exhaustion, queue pruning, and depth pruning. A batch can be continuing or completed; completion can be finished or truncated with one or more exact reasons.

The evidence layer remains backend-specific enough to be honest. Djinn can return validated candidates, a proof of uninhabitability for the translated problem, a self-reference-only diagnosis, or no evidence. Exference returns candidates and progress but never manufactures a negative proof from heuristic exhaustion.

4.7.1 Candidate values

synthesis/src/Language/Haskell/Synthesis/Candidate.hs packages a generated function clause with residual constraints and backend details. Backend details are intentionally extensible: Djinn can report proof/search information, while Exference can report steps, queue size, penalty, usage, and pruning statistics.

Candidate rendering revalidates scope. This may seem late, but it is a deliberate last defense before a neutral term crosses into source syntax. A candidate with an unbound local or an unresolved hole is not made printable merely because the search happened to construct it.

4.8 The generated-term language

The large synthesis/src/Language/Haskell/Synthesis/Generated.hs module is the common target of both backends. Its expression vocabulary includes locals, globals, holes during construction, application, visible type application, tuples, lambdas, lets, and cases. Patterns include binders, wildcards, tuples, declared constructors, and as-patterns. Function clauses associate checked definition names with expressions.

Several details are easy to miss:

- **Visible type application** is a real AST node. It is not a string annotation. This lets exact rank-N instantiation evidence survive into rendering.
- Application-spine helpers are strict about mixed term and type arguments. They preserve order and do not silently commute them.
- Scope validation understands every binder-introducing pattern, including nested tuple and constructor patterns.
- Simplification is capture-aware and must preserve the source of global versus local names.
- Qualification is postponed until rendering, so semantic identity and source spelling remain separate.

Invariant

The shared generated AST is neither Haskell source nor Lean source. It is a checked intermediate language. Backend code may construct it; frontend code may render it; neither side may attach semantics by concatenating unvalidated strings.

4.9 Diagnostics and selection

synthesis/src/Language/Haskell/Synthesis/Diagnostic.hs gives validation and search failures structured identities and renderings. The adapters use diagnostics instead of throwing ordinary exceptions for expected user

errors. Pure rendering paths still contain exception boundaries around hostile or infinite user-supplied text, but asynchronous exceptions are preserved.

`synthesis/src/Language/Haskell/Synthesis/Selection.hs` separates search generation from candidate selection. A backend may expose a lazy stream with scores and progress; a client chooses a bounded subset, filters it, and preserves the strongest progress observation. This is particularly important for Leant, which performs its own cross-engine merge and final verification window.

Chapter summary

The shared layer establishes the vocabulary and the facts that both engines are allowed to assume. Its most important contribution is not code reuse but semantic compression: downstream code receives opaque values that stand for completed validation arguments.

Chapter 5

Djinn: formulas, LJT proof search, and checked lowering

5.1 What Djinn promises

Djinn is the proof-theoretic backend. It translates a supported type problem into intuitionistic propositional logic, searches for proof terms in a contraction-free LJT-style calculus, checks every proof independently, and lowers accepted proofs into the shared generated syntax. When translation is complete and search is genuinely exhaustive, failure can be negative evidence. That last property is the reason Djinn's request preparation, translation plans, and progress accounting are more elaborate than a simple "type to formula" function.

The stable adapter is `djinn/src-core/Language/Haskell/Djex/Djinn.hs`. Its private request and session support live in `djinn/src-core/Language/Haskell/Djex/Djinn/Internal/Request.hs` and `djinn/src-core/Language/Haskell/Djex/Djinn/Internal/Session.hs`. The major engine entry points are in `djinn/src-core/Djinn/Core.hs`.

5.2 Request preparation

Preparation occurs in two layers.

5.2.1 Session-independent preflight

The adapter first validates facts that do not depend on a loaded environment: the target definition name, goal width, binder shape, explicit context width, option bounds, and finite observations. This stage turns a public `QueryRequest` into a normalized cached query or a structured diagnostic.

The benefit is deterministic failure. A malformed query should not fail differently depending on which session happened to receive it, nor should it force a large environment before reporting a simple width error.

5.2.2 Session-dependent elaboration

The checked session supplies declarations, classes, constructor inventories, kind information, synonyms, and source names. The private request builder:

- resolves and normalizes the goal against the prepared inventory;
- checks explicit class contexts against declared class arities;
- embeds contexts under the leading forall prefix in a representation that preserves scope;
- makes binder implicitness explicit to the compiler;

- verifies exact provider-instantiation assignments against the session;
- determines which loaded values and constructor schemas are eligible premises;
- excludes the target from the ordinary premise set.

Explicit contexts are validation obligations, not ordinary proof assumptions. A request for `C a => a -> a` should not be solved by treating the class dictionary as an arbitrary value of proposition `C a`; its role is to constrain the type and the admissible instances.

Failure mode to keep in mind

Reintroducing the target definition as a normal premise makes every recursive query trivially inhabitable. Djinn excludes it from the safe search environment. A separate diagnostic lane may reintroduce it only to establish that all observed proofs require self-reference.

5.3 The logical language

djinn/src-core/Djinn/Internal/LJTFormula.hs defines the formula and proof-term vocabulary. In simplified form:

```
data Formula
  = Conj [Formula]
  | Disj [(ConsDesc, Formula)]
  | Empty Symbol
  | Formula :-> Formula
  | PVar Symbol

data Term
  = Var ... | Lam ... | Apply ...
  | Ctuple ... | Csplit ...
  | Cinj ... | Ccases ... | Xsel ...
```

Products become conjunctions; algebraic alternatives become labeled disjunctions; functions become implication; empty nominal types and opaque atoms become symbols. Constructor descriptions retain enough information for proof lowering to recover data-constructor applications and case alternatives.

5.3.1 Symbols preserve nominal and opaque identity

A symbol can be an ordinary logical name or an opaque type symbol paired with source information and a `TypeAtomKey`. This prevents two unrelated empty nominal types from collapsing merely because each has no visible constructor. It also ensures that alpha-equivalent opaque polytypes compare correctly while distinct nested polytypes remain distinct.

This is essential for sound negative evidence. If the compiler identified all opaque regions with one proposition, it could invent implications between unrelated source types. If it generated a fresh proposition for each occurrence, it would lose valid forwarding of the same opaque type. The key must therefore be semantic and alpha-aware.

5.4 The prepared formula compiler

djinn/src-core/Djinn/Internal/TypeFormula.hs is the bridge between the shared type authority and the proof calculus. It is best understood as a compiler producing both code and a proof obligation ledger.

A prepared compiler caches validated definitions, constructor descriptions, synonym expansions, and recursive-component classifications. A translation result contains at least:

- the formula;

- whether the translation was incomplete;
- which positive forall sites were opened;
- which binders were skolemized;
- occurrence and origin metadata used to validate exact provider assignments;
- enough nominal information to lower a proof back to generated syntax.

5.4.1 One-layer views

The compiler does not recursively destruct every type by default. It uses a one-layer `TypeView`: observe the current constructor, decide whether that constructor is structurally admissible in the current polarity, and recursively compile only the selected children. This makes polarity and opacity policy explicit.

5.4.2 Forall polarity

A positive forall corresponds to producing a polymorphic value. Djinn can often open it by introducing a fresh rigid symbol. A negative forall corresponds to consuming a polymorphic value. Naively opening it would require impredicative or higher-rank reasoning beyond ordinary propositional translation. The conservative default is therefore:

- open selected positive forall sites;
- preserve negative and unsupported nested sites as opaque type atoms;
- add exact instantiation axioms only when the request carries checked evidence.

This is why rank-N support appears as a family of translation plans rather than a single recursive function.

5.4.3 Bounded plan families

For a query with several openable sites, the compiler explores a deterministic family of structural plans:

- all sites open;
- all sites opaque;
- selected singleton, pair, triple, quadruple, and quintuple frontiers in both open and opaque orientations;
- additional guarded plans tied to exact provider assignments and query-correlated closed instantiations.

The family is explicitly bounded. Quintuple combinations are capped per orientation, while remaining complete through a documented small number of sites. The cap is not hidden: if relevant plans were omitted, the translation carries incompleteness and a candidate-free search cannot become a refutation.

Why it is designed this way

Opening every positive forall is not always best. An opened site exposes structure that can enable a proof, but it can also destroy useful opaque equality: a value may simply need to be forwarded unchanged. The plan family searches both interpretations and deduplicates equivalent results afterward.

5.4.4 Recursive and nominal data

Recursive types require a similar duality. At a positive occurrence, the compiler may expose one constructor layer to synthesize or inspect data. At the same time it preserves an exact opaque fallback so forwarding remains possible without infinite unfolding. Recursive components are classified before compilation, and every exposure decision contributes to the completeness ledger.

Nominal parametric data is transported with constructor descriptions and parameter metadata. The compiler must not substitute a structurally similar but differently named type. Exact constructor identity is carried into proof terms and later lowering.

5.4.5 Exact provider-assignment retention

Provider assignments are only useful if the formula still represents the exact argument vector that was validated at the source boundary. The compiler records origins and occurrence paths and checks that a structural plan retains the assignment across aliases and higher-kinded application. A plan that has hidden or rearranged a relevant argument cannot consume the assignment merely because its pretty-printed formula looks compatible.

5.5 LJT search

The proof search implementation is in `djinn/src-core/Djinn/Internal/LJT.hs`. It combines a contraction-free sequent calculus with a continuation-encoded lazy search monad.

5.5.1 Search modes

A search mode chooses an exploration strategy and an optional global choice-point budget. The principal strategies are depth-first exploration and fair interleaving. The default unbudgeted mode is intended to retain the calculus's completeness properties, subject to documented local fairness adjustments.

The result stream is lazy. A client may consume the first proof without constructing every proof. At the same time, budget and exhaustion information remain observable. Internally, the search distinguishes:

- a completed branch;
- a yielded proof with a continuation;
- an administrative step that advances the proof machine without producing a proof.

This distinction permits exact choice-point accounting and fair merges without forcing the whole tree.

5.5.2 Antecedent classification

LJT gains much of its behavior by classifying antecedents. The implementation separates pending formulas, atomic implications, nested implications, and already available atomic proofs. Right-reduction introduces functions, products, or disjunction choices. Left-reduction selects an assumption and applies the appropriate elimination rule.

The calculus is contraction-free in its structural formulation: it avoids arbitrary duplication as a primitive rule. Where a proof term must use an assumption more than once, the formula-specific left rule accounts for it. This keeps the search space much smaller than a naive natural-deduction enumerator.

5.5.3 Cheap reachability pruning

A necessary-condition test, conceptually `goalMayBeReachable`, rejects branches whose atomic goal symbols cannot arise from the current antecedents. This is not a theorem prover by itself; it is a conservative prune. Returning false must mean the branch is impossible. Returning true merely allows full proof search to continue.

5.5.4 Fairness for repeated endomorphisms

Types with several endomorphism assumptions can produce infinite local families such as `f x`, `f (f x)`, and so on. Pure depth-first exploration can starve sibling compositions. The implementation applies a local round-robin or interleaving policy for repeated binary endomorphism opportunities. Freshness and substitution machinery ensure that copied proof fragments do not capture binders.

5.5.5 Normalization

Proof terms are normalized before checking and lowering. The normalizer beta-reduces lambda applications, tuple split redexes, and case/injection redexes. Normalization reduces duplicate candidates and gives the proof checker a simpler object. It must remain binder-safe: every copied binder is freshened before substitution.

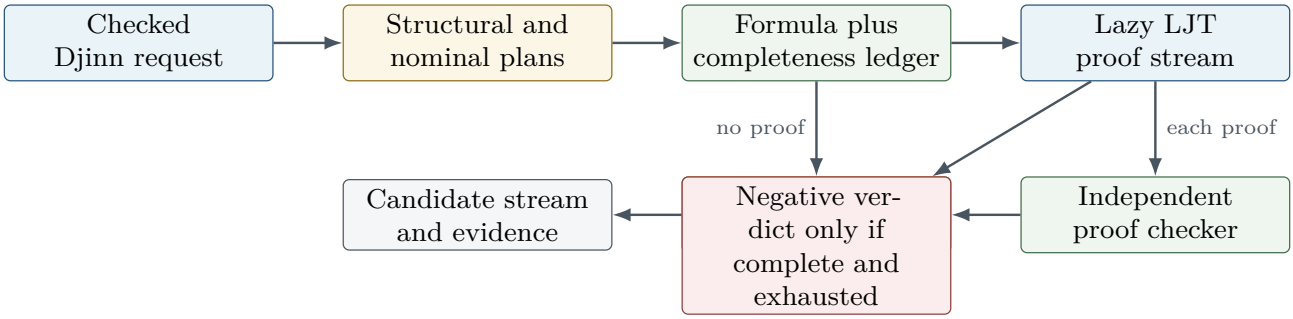

Figure 5.1: Djinn's proof-producing pipeline. The negative edge is guarded by both compiler completeness and search exhaustion.

5.6 Core orchestration and instantiation axioms

`djinn/src-core/Djinn/Core.hs` orchestrates plan construction, premise selection, LJT execution, proof validation, deduplication, ranking, and evidence. The file is large because it is where independent correctness conditions meet.

The engine can derive several classes of controlled instantiation premise:

- closed monotype instantiations available from the query itself;
- query-correlated instantiations whose shared variables must receive one coherent assignment;
- loaded polymorphic values whose exact schemes are known in the session;
- provider-local assignments supplied by Leant or another client;
- visible type-application evidence retained in the generated term.

These are not unconstrained axioms. Each is created from a checked source scheme, exact argument vector, kind proof, and plan-retention check. After the proof checker validates a derivation in the enriched proof environment, the instantiation evidence is rewritten or erased in a controlled lowering step so the generated term refers to actual source values and visible type applications rather than fictional logical constants.

5.7 Proof checking and lowering

Every LJT proof is passed to `djinn/src-core/Djinn/Internal/ProofCheck.hs`. The checker reconstructs the derivation against the formula and proof environment. Search ordering, cached reachability, and heuristic plan choice are not trusted as proof evidence.

Accepted proofs are converted by `djinn/src-core/Djinn/Internal/ProofToGenerated.hs`. The lowering pass maps logical introductions and eliminations to lambda, application, tuple, constructor, and case syntax; restores source constructor identities; resolves instantiation witnesses; and produces a shared generated clause. Scope is checked again at the shared AST boundary.

Invariant

Djinn's trusted result is not "the search code said success." It is "the independent checker accepted this proof term for this translated formula, and the lowering pass produced a scoped generated term." The final client may still need to check that translation and rendering preserve the source-language goal.

5.8 Evidence outcomes

Conceptually, the core can distinguish:

- **realized**: one or more checked candidates exist;
- **unrealizable**: exhaustive complete search found no proof;
- **unrealizable without self-reference**: a diagnostic run proves that only reintroducing the target enables a proof;
- **undecided**: a budget, incomplete plan family, opaque unsupported structure, or other limitation prevents a verdict.

A global candidate cutoff is shared across plans. To know whether the cutoff actually truncated a nonempty tail, the implementation observes one extra proof as a witness. Choice-point budgets are likewise shared across plan execution rather than reset per plan. Otherwise a caller requesting a budget of N could accidentally receive N work for every structural plan.

Candidates from different plans are deduplicated after normalization, including structural and eta-equivalent forms where the shared representation can establish equivalence. Optional ranking favors compact terms and can consider unused-binder ratios.

5.9 Worked trace: identity

For `forall a.  a -> a`, a representative trace is:

1. Request preparation validates one leading forall and no class context.
2. A positive-forall plan opens `a` as a fresh rigid proposition symbol.
3. The compiler emits $a \to a$.
4. Right implication introduction extends the antecedent with $x : a$ and sets goal a.
5. The atomic goal is discharged from the antecedent, yielding a variable proof.
6. The proof is wrapped in a lambda, normalized, and checked.
7. Lowering emits a lambda over one local binder.
8. The plan family may also consider an opaque interpretation, but deduplication removes equivalent forwarding terms.

The apparent triviality is misleading. The same machinery must retain alpha identity if the forall is nested, avoid opening it at a negative polarity without evidence, and keep the binder rigid in the proof environment.

5.10 Worked trace: swapping a product

For `(a,b) -> (b,a)`, the compiler emits a conjunction implication. Search introduces the input, splits it into two components, constructs a conjunction in reverse order, and returns a proof term using tuple split and tuple construction. Lowering chooses a generated pattern/case representation. Later frontends may render it as a tuple-pattern lambda or as an explicit match depending on target-language constraints.

This trace is a useful debugging probe because it crosses both an elimination and an introduction, exercises binder scopes created by a destructuring pattern, and reveals whether normalization preserves constructor descriptions.

5.11 Debugging Djinn

When Djinn unexpectedly returns no term, inspect in this order:

1. Was the public query rejected or was a checked request actually constructed?
2. Which type regions became opaque, and did the compiler mark translation incomplete?
3. Which structural plans were generated and which exact assignments survived each plan?
4. Were relevant loaded values or constructors omitted by premise policy?
5. Did the LJT stream exhaust, hit a choice-point budget, or hit the candidate cutoff?
6. Did a proof appear but fail `ProofCheck`?
7. Did lowering or generated-scope validation reject it?
8. Did client-side rendering or final source elaboration reject the neutral candidate?

The distinction between steps 5, 6, and 8 is particularly important. A proof-search defect, a proof-checker rejection, and a Lean elaboration rejection require entirely different fixes.

Source trail

The shortest source trail through a Djinn request is: `djinn/src-core/Language/Haskell/Djex/Djinn.hs` → `djinn/src-core/Language/Haskell/Djex/Djinn/Internal/Request.hs` → `djinn/src-core/Djinn/Core.hs` → `djinn/src-core/Djinn/Internal/TypeFormula.hs` → `djinn/src-core/Djinn/Internal/LJT.hs` → `djinn/src-core/Djinn/Internal/ProofCheck.hs` → `djinn/src-core/Djinn/Internal/ProofToGenerated.hs`.

Chapter 6

Exference: typed-hole search, ranking, and reconstruction

6.1 What Exference promises

Exference is the heuristic and ranking-oriented backend. It searches a state space of partially constructed typed expressions. Nodes contain typed holes, lexical scopes, rigid and flexible type variables, class obligations, globals, exact provider evidence, constructor/deconstructor knowledge, usage counts, freshness supplies, and a priority score. The scheduler expands the most promising node, bounds the queue and depth, and emits solved expressions. An independent checker reconstructs each candidate before it crosses the adapter boundary.

Exference is often better than Djinn when a useful term should call a library value, when several plausible terms need ranking, or when rank-N elimination and explicit type applications are involved. Its negative result is intentionally weaker: it does not claim that queue exhaustion proves uninhabitability.

The adapter starts at `exference/src-core/Language/Haskell/Djex/Exference.hs`; the central scheduler is `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs`; node data and construction are in `exference/src-core/Language/Haskell/Exference/Core/Internal/ExferenceNode.hs` and `exference/src-core/Language/Haskell/Exference/Core/Internal/ExferenceNodeBuilder.hs`.

6.2 Checked environment and request

The adapter builds an opaque Exference session from the shared environment. It projects declarations, class information, constructor/deconstructor bindings, ratings, visibility, and prepared schemes into Exference-specific authorities. Projection can omit unsupported declarations, but omissions are returned as diagnostics rather than silently changing the query.

A checked Exference query seals together:

- the exact prepared environment;
- the normalized goal and explicit contexts;
- global and local source-name mappings;
- provider candidates and exact assignments;
- a rigid-instantiation plan;
- search options and penalty overrides.

Assignment validation checks exact provider scheme identity, argument count, absence or treatment of contexts, kinds, closure, finite widths, and duplicates. A textual name match is insufficient.

6.3 Types and unification

The core reuses the shared neutral type through compatibility patterns. `exference/src-core/Language/Haskell/Exference/Core/Types.hs` defines Exference-facing views, substitutions, polytype helpers, and scope-sensitive operations. The actual unifier is `exference/src-core/Language/Haskell/Exference/Core/Unify.hs`.

6.3.1 Disjoint and shared namespaces

Exference needs several unification modes. Two schemes may use the same integer identifiers but mean different variables; a local expression and a global scheme may share a rigid scope; a right-hand pattern may contain the only variables permitted to bind. The API therefore distinguishes disjoint, shared, and one-sided unification rather than relying on caller folklore.

Internally, variables are tagged with their origin before unification. This prevents accidental capture or identification merely because two integer supplies both started at zero.

6.3.2 Higher-kinded heads

The unifier observes a constructor-application form. Functions and tuples can be lowered to canonical heads for this purpose, so a flexible head variable can bind to a type constructor of the right kind. Kind checking is not deferred until rendering: a substitution that solves the first-order shape but violates kind arity is rejected.

6.3.3 Opaque polytypes

Nested polytypes become tagged opaque atoms. They can unify as complete values when their alpha-normal keys and substitutions agree, but the unifier cannot decompose a forall under an application as if it were a monotype. Occurs checks still inspect the atom's free variables. Substitution into the atom is capture-avoiding.

Invariant

A nested forall is simultaneously opaque to structural decomposition and transparent to lexical hygiene. "Opaque" never means "free-variable blind."

6.4 Classes and residual constraints

Exference builds a static class environment from validated class and instance declarations. It checks duplicate or overlapping instance heads according to the supported policy, class arity, superclass cycles, and recursively expanding prerequisites. Search may carry scoped givens and unresolved obligations. A solved expression is only a candidate after constraint closure determines which residual constraints are valid and whether they escape the permitted scope.

The expression checker later recomputes residual constraints and compares them with the candidate's claimed set. Search cannot simply attach a convenient list.

6.5 The search node

A node is a complete snapshot of one partial program. Important fields include:

- pending typed holes and the already selected next goal;
- a scope forest describing lexical parentage;
- local givens and scoped class constraints;
- flexible substitutions and rigid scopes;

- global bindings with exact schemes and source identities;
- visible type-application candidates;
- provider candidate and assignment maps;
- constructor and deconstructor bindings;
- per-binding usage counts and donation guards;
- the current annotated generated expression;
- fresh supplies for term variables, type variables, scopes, and holes;
- accumulated depth, last selected binding, and score inputs.

The apparent duplication is deliberate. Search branches must be clonable and self-describing. A node should not depend on mutable ambient state whose meaning changes after it is enqueued.

6.5.1 Typed goals

A pending goal records a hole identifier, expected type, lexical scope, forall-introduction mode, tuple policy, and local givens. Forall goals can be in phases such as opening an existing leading prefix, trying an introduction, or continuing a previously selected introduction. Tuple goals can choose a shallow structural lane or a whole-tree shortcut. These modes prevent one generic expansion rule from repeatedly rediscovering the same administrative choices.

6.5.2 Freshness and identifier exhaustion

Node construction uses explicit finite supplies. If a fresh identifier cannot be allocated within the supported space, the branch records identifier exhaustion and terminates cleanly. Wraparound is never allowed to produce aliasing between an old binder and a new hole.

6.6 The scheduler loop

The scheduler maintains a maximum-priority queue of nodes. A scheduled node already has its next goal selected, so the queue ordering is over actionable states rather than states that must perform another non-deterministic pop after dequeue.

One iteration conceptually does the following:

```
step state =
  case popBest state.queue of
    Nothing   -> finishWithExactPruningEvidence state
    Just node ->
      let children = expandSelectedGoal node
          (tooDeep, admissible) = partitionDepth children
          (solved, future)      = partitionSolved admissible
          checked              = checkSolved solved
          scored               = score future
          queue'               = mergeWithCapacity scored state.queue
      in emit checked (updateCounters queue' tooDeep)
```

Again, this is explanatory pseudocode. The implementation is careful about laziness, exact prune counts, and continuation state.

6.6.1 Queue capacity and exact pruning

A bounded queue keeps the search operational in large libraries. The merge routine must retain the best nodes under the ordering while reporting exactly how many were discarded. It cannot append two queues, truncate, and then guess which source lost nodes. Those exact counts become `QueueLimitPruned` and contribute to progress.

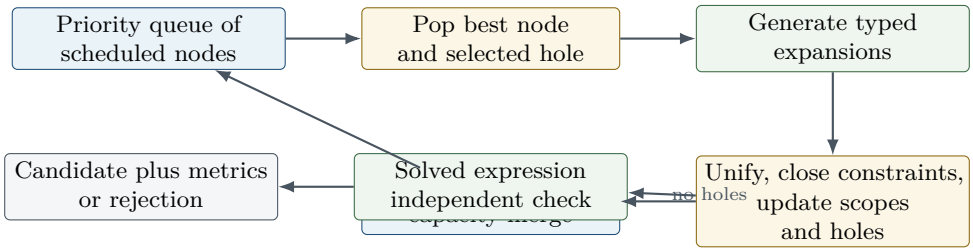

Figure 6.1: The Exference search loop. Queue pruning and depth pruning are measured, not inferred after the fact. Solved nodes bypass the queue and enter a separate checking path.

Depth pruning is recorded separately. A final empty queue after any pruning means "no remaining node under this policy," not "the type is impossible."

6.6.2 Expansion families

Depending on the goal, expansion can:

- introduce a lambda binder;
- open or preserve a forall;
- choose a local or global binding and create argument holes;
- insert visible type applications from exact evidence;
- construct a tuple or data constructor;
- eliminate a tuple or data value through a case;
- use a provider with an occurrence-local assignment;
- introduce a let or reuse a previously built value;
- carry or discharge class obligations.

Every expansion updates the expression and the type state together. A term edge without the corresponding substitution, scope, and hole changes is not a valid child node.

6.7 Scoring and usage

The score combines structural complexity, configured binding ratings, type complexity, depth, class burden, provider penalties, usage policy, and other heuristic features. Source ratings are inputs, not proof evidence. Leant can supply relevance-biased overrides for providers discovered from the current environment.

Usage counts serve several purposes. They can prefer candidates that use requested arguments, reject candidates with unused binders under a strict lane, and provide backend metrics. The strict and relaxed policies are often run as separate lanes rather than one score penalty, because "unused forbidden" is a semantic filter while "unused discouraged" is a heuristic.

6.8 Expression representation

`exference/src-core/Language/Haskell/Exference/Core/Expression.hs` wraps the shared generated syntax with annotated locals carrying internal identifiers and optional type information. Compatibility pattern synonyms preserve older Exference-facing constructors while the storage representation converges on the shared AST.

The conversion boundary erases search-only annotations only after scope and typing checks. This is another example of keeping a richer internal proof object until the last responsible moment.

6.9 The independent expression checker

`exference/src-core/Language/Haskell/Exference/Core/ExpressionCheck.hs` is the key trust boundary. It does not accept the search node's accumulated typing story as a derivation. It reconstructs the expression bidirectionally against an opaque prepared checker context.

The checker validates:

- generated-tree well-formedness and absence of holes;
- local binder scope and global source identity;
- applications, lambdas, tuples, constructors, lets, and cases;
- nested forall introduction and visible type application;
- rigid-scope provenance and absence of escaping rigids;
- constructor/deconstructor schemes;
- class givens and residual obligations;
- exact equality between reconstructed and claimed residual constraints.

It reuses the pure unification kernel because duplicating unification would create two semantics. It does not reuse the search scheduler, queue ordering, score, or node-local assumption that a hole had a certain type. Search contributes only irreducible provenance that cannot be recovered from the erased expression, such as the origin of certain nested rigid instantiations.

The simplified candidate is checked first. If simplification changed a representation in a way the checker rejects, the raw expression may be checked as a fallback. A candidate that fails both is dropped with diagnostics; it is not exposed as "probably typed."

Why it is designed this way

The independent checker turns Exference from a heuristic term generator into a checked heuristic term generator. The search can be aggressively optimized or reordered without making every optimization part of the trusted typing argument.

6.10 Adapter output and defensive rendering

The stable adapter converts checked internal candidates to shared candidates and attaches metrics such as steps, penalty, final queue size, binding usage, and exact pruning totals. Residual constraints and generated scope are validated again.

Diagnostic hints can originate in user or source text and may be cyclic or infinite in a lazy language. Pure rendering observes a bounded prefix and catches synchronous exceptions while preserving asynchronous exceptions. A bad hint must not take down an otherwise valid search result.

6.11 Worked trace: applying a provider

Consider a goal corresponding to `(a -> b) -> F a -> F b` with a provider `mapF : forall x y. (x -> y) -> F x -> F y`.

1. The root forall prefix creates rigid type variables `a` and `b`.
2. Arrow introduction puts `f : a -> b` and `xs : F a` in scope.
3. The result hole expects `F b`.

4. Selecting `mapF` instantiates its two quantified variables. Exact provider evidence correlates `x=a` and `y=b` at this occurrence.
5. The instantiated result unifies with `F b`; two argument holes are created for `a -> b` and `F a`.
6. Local selection fills them with `f` and `xs`.
7. The solved expression contains a global provider occurrence plus visible type applications when required by the source scheme.
8. The independent checker reconstructs the scheme instantiation and local applications.

This trace exercises the features for which Exference's rich state is justified: occurrence-local assignments, explicit instantiation, global/local scope, and ranking of a library-oriented term.

6.12 Why Exference does not refute

Even a completely empty final queue may be the result of:

- a finite step limit;
- queue-capacity pruning;
- depth pruning;
- a finite identifier supply;
- disabled expansion families;
- ranking that starved a productive region before the limit;
- an intentionally incomplete provider or declaration projection;
- candidate policies such as forbidding unused binders.

The adapter reports these conditions precisely. It never upgrades them to a logical proof. Leant's combined mode preserves this asymmetry: an Exference miss cannot cancel or create a Djinn refutation.

6.13 Djinn and Exference compared

Dimension	Djinn	Exference
Internal object	Formula, sequent state, proof term	Typed expression with holes, scopes, substitutions, queue state
Primary strength	Structural/proof-theoretic synthesis and conditional negative evidence	Ranked library/provider-oriented synthesis and richer elimination/instantiation
Ordering	Calculus strategy and plan family	Priority score with queue/depth limits
Independent validation	Proof checker	Bidirectional expression checker
Rank-N treatment	Opaque-by-default formula plans plus checked instantiation axioms	Opaque polytypes, rigid scopes, visible type applications, occurrence-local assignment
Negative result	Possible when translation and search are complete	Never promoted to proof of uninhabitability
Candidate ranking	Optional, comparatively light	Core design feature with ratings and usage metrics

Dimension	Djinn	Exference
Typical failure question	“Was translation/search complete?”	“Which policy, score, or bound removed the productive node?”

Source trail

The shortest Exference source trail is: `exference/src-core/Language/Haskell/Djex/Exference.hs` → `exference/src-core/Language/Haskell/Djex/Exference/Internal/Request.hs` → `exference/src-core/Language/Haskell/Exference/Core/Internal/ExferenceNodeBuilder.hs` → `exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs` → `exference/src-core/Language/Haskell/Exference/Core/Unify.hs` → `exference/src-core/Language/Haskell/Exference/Core/ExpressionCheck.hs`.

Chapter 7

The unified facade, source frontends, and REPL

7.1 A capability-aware facade

src/Language/Haskell/Djex.hs is the preferred programmatic entry surface. It defines the backend selection, common session/query operations, and an explicit capability vocabulary. Capabilities include deciding inhabitation, heuristic search, prenex polymorphism, rank-N introduction and elimination, opaque rank-N transport, impredicative support, ranked candidates, and type-class constraints.

The capability matrix is executable documentation. A client can choose Djinn when a complete structural negative result matters, Exference when ranking or elimination matters, or both when it can merge candidates while respecting asymmetric evidence.

Invariant

A facade may normalize calling conventions, but it may not normalize away semantic differences. "No candidates" has backend-specific meaning and remains so after dispatch.

7.2 Source conversion

src/Language/Haskell/Djex/HaskellSrc.hs and the historical frontend conversion modules parse Haskell source declarations into the shared environment vocabulary and render generated terms back to Haskell-like source. The conversion boundary handles qualification, operator names, source locations, fixities where supported, class declarations, and the mismatch between source syntax and neutral AST structure.

The historical Exference frontend under `exference/src-frontend/Language/Haskell/Exference` includes environment parsing, declaration extraction, source-type conversion, class-environment construction, expression rendering, and CLI support. The historical Djinn frontend under `djinn/src-frontend` retains its original command vocabulary and help surface.

These frontends are useful compatibility layers, but new semantic changes should enter through shared authorities and checked adapters. Adding a type feature only to a source parser creates a value neither backend has promised to understand.

7.3 Packages, workspaces, and dependency order

src/Language/Haskell/Djex/Package.hs and src/Language/Haskell/Djex/REPL/Workspace.hs manage packages and workspaces. A workspace is not merely a list of files. It must resolve module identities, dependencies,

declaration order, loaded environments, and command-line tool interaction. src/Language/Haskell/Djex/Internal/DependencyGraph.hs supplies dependency ordering and cycle handling.

Package discovery can invoke Cabal-facing tooling. Test support includes a fake Cabal executable so CLI behavior can be tested without making unit tests depend on a user's machine configuration.

7.4 The unified REPL

The large src/Language/Haskell/Djex/REPL.hs coordinates the interactive session. Focused submodules divide concerns:

Module	Responsibility
src/Language/Haskell/Djex/REPL/Command.hs	Command grammar, abbreviations, arguments, and command-level diagnostics.
src/Language/Haskell/Djex/REPL/Driver.hs	Input loop, prompt, multiline behavior, output and exception boundary.
src/Language/Haskell/Djex/REPL/Scope.hs	Visible declarations, qualification, ambiguity, namespace resolution, and import/export effects.
src/Language/Haskell/Djex/REPL/Type.hs	Type parsing, normalization, checking, and user-facing type operations.
src/Language/Haskell/Djex/REPL/Kind.hs	Kind queries and diagnostics.
src/Language/Haskell/Djex/REPL/Eval.hs	Evaluation-oriented commands and source execution boundaries.
src/Language/Haskell/Djex/REPL/DjinnScope.hs	Projection of the unified visible scope into Djinn-specific premise policy.
src/Language/Haskell/Djex/REPL/Workspace.hs	Workspace loading, module graph, package context, and reload behavior.

The top-level REPL maintains both source-facing state and backend sessions. A declaration update must rebuild or increment every dependent authority in a defined order. The REPL should never retain an Exference session built from environment version n while rendering against source scope version $n + 1$.

7.5 Commands and the CLI

src/Language/Haskell/Djex/Command.hs defines shared command structures, while src/Language/Haskell/Djex/CLI.hs handles process arguments, modes, files, and startup behavior. src/Language/Haskell/Djex/Text.hs centralizes bounded text helpers and presentation details.

The three executables are intentionally thin at their `app/Main.hs` entry points. Substantive process behavior lives in library modules, which is why CLI tests can call command logic without forking for every assertion while still retaining end-to-end executable tests.

7.6 Extension discipline

A safe feature generally lands in this order:

1. add or refine a shared authority and its pure tests;
2. update environment/inventory validation and kind inference;
3. teach Djinn's adapter/compiler only what it can soundly translate;
4. teach Exference's adapter/unifier/search/checker only what it can reconstruct;

5. update the generated AST if a genuinely new term form is required;
6. update source frontends and renderers;
7. expose a capability bit or diagnostic when support is asymmetric;
8. add facade, REPL, CLI, and integration tests.

Starting at the REPL and working downward usually creates a period in which text syntax exists without a validated semantic path.

Chapter 8

Djex tests, benchmarks, and diagnostic workflow

8.1 Concentric test layers

The repository's test organization mirrors its trust boundaries.

Suite	What it protects
synthesis/test/Spec.hs	Shared names, types, substitution, alpha equivalence, environments, kind inference, queries, generated syntax, inventories, finite-width guards, and diagnostics.
djinn/test-unit/Spec.hs	Formula translation, LJT rules, normalization, instantiation plans, proof checking, lowering, evidence, budgets, and regressions.
djinn/test-property/PropertySpec.hs	Property-level relationships such as checker acceptance, normalization, and generated-term invariants over broad inputs.
exference/test-unit/Spec.hs	Unification, rigid scopes, class closure, node expansion, scoring, provider assignments, expression checking, pruning, and regressions.
exference/test-engine/EngineSpec.hs	Black-box engine behavior with checked environments and bounded search.
test-api/AbstractionBoundary.hs	Compile-time enforcement that internal constructors and modules do not leak through the supported public API.
test-api/FacadeSpec.hs	Backend dispatch, capability reporting, common query behavior, and facade diagnostics.
test-integration/Spec.hs	Cross-layer behavior: source declarations through sessions and engines to rendered candidates and evidence.
test-cli/Spec.hs	Unified command-line and REPL protocol, formatting, files, workspaces, package discovery, and failure behavior.
Historical CLI/API suites	Compatibility of the retained `djinn` and `exference` entry surfaces.

The abstraction-boundary tests deserve special attention. In Haskell, a module export list is part of the correctness story. If a constructor becomes public accidentally, downstream code can fabricate a value that bypasses every runtime test. Compile-time negative tests guard that boundary.

8.2 Benchmarks

Both historical engines have benchmark corpora. A useful benchmark change should record more than wall-clock time. For Djinn, track proof-plan counts, choice points, proof prefix latency, and deduplication. For Exference, track steps, peak/final queue size, exact queue/depth pruning, candidate rank, and checker rejection. A speedup that changes the first accepted candidate or turns finished search into truncation is a semantic change and should be reviewed as such.

8.3 A practical failure funnel

When a top-level test says "expected candidate, got none," localize the loss:

1. **Source conversion**: did the declaration or goal parse to the intended shared type?
2. **Environment**: did validation retain the declaration and indexes?
3. **Projection**: did the selected backend session include the value, constructor, class, or instance?
4. **Request**: did exact contexts and provider assignments survive checked preparation?
5. **Search**: did the backend generate a proof/node, or did a bound/prune intervene?
6. **Internal checker**: did proof or expression reconstruction reject it?
7. **Shared AST**: did scope, simplification, or candidate conversion reject it?
8. **Frontend**: did qualification or source rendering reject or duplicate it?

Instrumenting all eight at once produces noise. Add a probe at the earliest boundary whose output is wrong.

8.4 Negative-evidence tests

The most valuable negative tests are not examples of impossible types; they are examples where a tempting implementation would falsely claim impossibility. Include cases with:

- a search budget reached just before a proof;
- an omitted rank-N structural plan;
- an opaque nominal type that must be forwarded;
- a recursive type whose one-layer exposure is incomplete;
- a provider assignment that a plan fails to retain exactly;
- target self-reference excluded from the safe environment;
- Exference queue/depth pruning followed by an empty queue.

These tests protect the distinction between "no proof exists" and "this implementation did not find one."

Chapter summary

Djex's test suite is best read as a map of trusted boundaries. Pure shared tests establish authorities; backend tests establish search and checking; API tests establish encapsulation; integration and CLI tests establish that the boundaries compose.

Part III

Leant

Chapter 9

Repository and runtime architecture

9.1 Build graph and reproducible pairing

Leant is a Haskell executable package described by `leant.cabal`. It depends on Djex as a library, Haskeline for the interactive terminal, process and directory APIs for the Lean backend, and a deliberately small set of utility packages. `cabal.project` builds the Leant package together with `lib/Djex`. The latter is a Git submodule declared in `.gitmodules` and pinned, at this snapshot, to the exact Djex commit used throughout Part II.

That pin is architecturally significant. The bridge imports internal details of the shared generated and query vocabulary that evolve in tandem with provider evidence and rank-N support. A floating package dependency would make it difficult to know whether a candidate-rendering failure came from Leant or an incompatible Djex revision.

The executable is launched directly or through `leant.cmd` on Windows. The Lean side is not linked into the Haskell process. Leant discovers a Lake project and a Lean REPL executable, then starts that backend as a child process.

9.2 The source is intentionally concentrated

The principal modules are:

Module	Responsibility
`src/Main.hs`	Entry point, command dispatcher, interactive state, ordinary Lean command execution, imports, files, undo, reset/reload, generated `it` bindings, prove mode, synthesis command orchestration, verification, and user-facing output.
`src/Leant/Backend.hs`	Backend discovery, child-process lifecycle, UTF-8 handles, request framing, timeouts, server death, bounded stderr capture, and low-level errors.
`src/Leant/Json.hs`	Dependency-light JSON AST, parser, and encoder used by the backend protocol.
`src/Leant/Classify.hs`	Lexical command classification and multiline continuation heuristic; it is deliberately not the Lean parser.
`src/Leant/Format.hs`	Source-like formatting of selected `##print` output.
`src/Leant/Builtins.hs`	Static help for Lean syntax and keywords that are not ordinary environment constants.
`src/Leant/Session/Replay.hs`	Pure transactional replay accumulator for environments, history, undo stack, and generated `it` counters.
`src/Leant/Session/Snapshot.hs`	Snapshot metadata, fingerprinting, companion artifacts, validation, durable temporary ownership, and atomic publication.
`src/Leant/Synth/Fragment.hs`	Lean payload serializer source, S-expression parser, `Frag` grammar, provider metadata, fragment analyses, recursive-family completeness, and classical/structural transformations.

Module	Responsibility
src/Leant/Synth/Engine.hs	Fragment-to-Djex translation, Djinn and Exference sessions, provider assignments, library premises, staged engine runs, candidate merging, evidence policy, and bounded forcing.
src/Leant/Synth/Render.hs	Neutral generated-AST fitting to exact Lean binders, premise removal, provider replacement, pattern normalization, naming, textual variants, and Lean source rendering.
src/Leant/Synth/ProviderCache.hs	Semantic provider query keys, provider-world generations, and bounded recency cache.
src/Leant/Synth/Replay.hs	Pure decision whether a derived synthesis environment can be reused, replayed by suffix, or rebuilt.

The architecture is asymmetrical: the pure and trust-sensitive transformations are factored into focused modules, while orchestration remains concentrated in `Main.hs`. This makes a top-down read feasible—almost every command begins in one file—but it also means changes to state ownership must be made carefully. A future refactor can extract command families and synthesis orchestration without moving the pure fragment/engine/render boundaries.

9.3 Runtime components

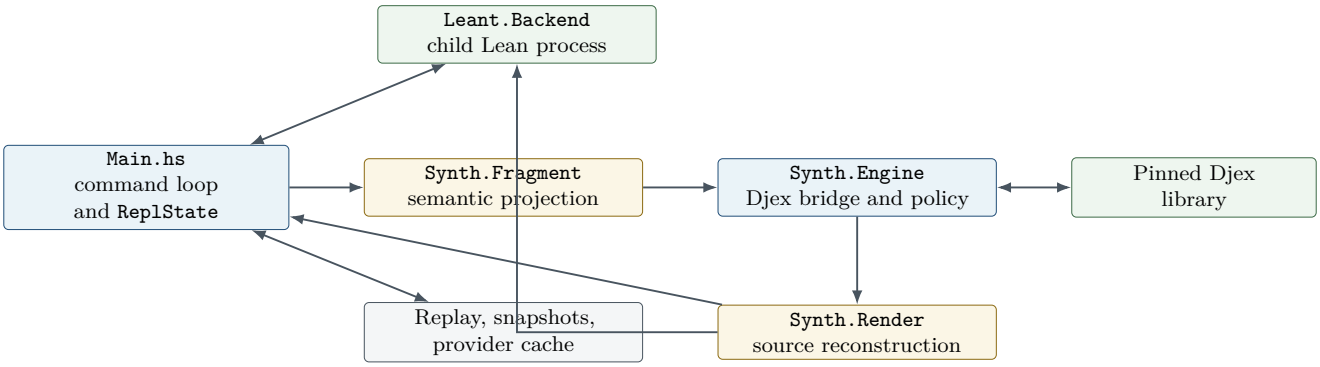

Figure 9.1: Leant's principal module/runtime relationships. The child Lean process is used at both ends: to project the goal and to verify reconstructed candidates.

9.4 Startup

A representative startup path in `Main.hs` is:

1. normalize console and buffering behavior; on Windows, request UTF-8 code pages and virtual-terminal support;
2. inspect command-line arguments and environment overrides;
3. discover the current Lake project by walking ancestor directories;
4. discover the Lean REPL executable, including the LeanInteract cache convention;
5. create a `BackendConfig`;
6. start the backend lazily or eagerly according to the command path;
7. establish the initial Lean environment and baseline imports;
8. initialize Haskeline and the large `ReplState`;
9. enter the command loop.

The backend may die later. Startup is therefore not a one-time phase; the same discovery/configuration data supports replacement processes and transactional reconstruction.

Chapter 10

The Lean backend protocol and small support modules

10.1 Backend configuration and discovery

src/Leant/Backend.hs defines a configuration containing the Lake executable or path, the Lean REPL executable, and the working directory/project. Discovery respects explicit environment configuration before probing caches and ancestors. The command is assembled as a Lake environment invocation so the backend sees the exact package search path and toolchain selected by the project.

The working directory matters. Running the same binary outside the Lake root can change import resolution, output artifacts, and environment fingerprints. Leant carries the directory in configuration rather than relying on a mutable process-global current directory.

10.2 Process creation

The child process is created with piped standard input, output, and error. The handles are configured for UTF-8 and line buffering. Backend state owns the process handle and all three pipes. Ownership is singular: when a request times out or the server closes, Leant terminates and discards the whole backend value rather than leaving a half-valid handle set in the main state.

The stderr pipe is not merged with stdout because stdout is a framed data protocol. On server death, stderr is drained through a bounded routine: a maximum line count and short per-line timeout prevent diagnostics from hanging reconstruction forever.

10.3 Request framing

Leant sends one JSON object followed by a blank line and flushes. It reads response lines until a blank line, joins them, and parses the result as JSON. The blank-line delimiter lets the Lean backend emit formatted JSON over several lines without requiring a binary length prefix.

The framing layer distinguishes three principal failures:

- `ServerClosed`: the child exited or closed stdout; bounded stderr is attached when available;
- `RequestTimeout`: the configured wall-clock request limit elapsed;
- `BadResponse`: a complete frame arrived but was not valid JSON or did not have the expected structure.

A timeout is treated more severely than an ordinary Lean error in the JSON payload. The process may still be computing or may have emitted a partial frame, so Leant kills it and marks the session for replay on the next command.

Invariant

A backend process and a Lean environment identifier form one lease. After timeout or transport death, no old environment identifier may be sent to a replacement process. Reconstruction must establish new identifiers in the new process first.

10.4 The local JSON implementation

`src/Leant/Json.hs` defines a compact JSON value type and hand-written parser/encoder. It supports objects, arrays, strings, numbers, Booleans, and null; string parsing handles escapes, Unicode escapes, and surrogate pairs.

Keeping JSON local has two advantages. First, the protocol surface stays explicit and small; error messages can mention the exact expected shape. Second, snapshot and interactive behavior are not coupled to a large generic encoding layer whose defaults might change field order, number handling, or duplicate-key policy.

The module should be treated as protocol infrastructure. Changes need round-trip tests, malformed-input tests, escaped-control tests, and non-BMP character tests. A permissive parser can be dangerous here: accepting trailing garbage may desynchronize frame handling.

10.5 Lexical classification, not parsing

`src/Leant/Classify.hs` answers practical questions before sending text to Lean: Does the line begin a top-level declaration? Are delimiters balanced? Does the user likely intend another line? It strips or ignores string literals, character literals, line comments, and nested block comments while scanning.

It does not decide whether Lean syntax is valid. The backend remains the parser authority. The classifier is allowed to be conservative: asking for one extra continuation line is inconvenient; prematurely sending half a declaration is destructive to interactive flow.

The top-level keyword set includes declaration forms and command prefixes used to classify history and provider-world invalidation. Any new Lean syntax added here should be tested inside comments and strings so false positives do not alter state.

10.6 Formatting and built-in help

`src/Leant/Format.hs` reshapes selected `##print` output into readable Lean-style declarations. It recognizes inductive, structure, and class layouts and indents definitions. This is presentation logic over backend output, not a parser used for synthesis.

`src/Leant/Builtins.hs` fills a different gap. Lean keywords and syntax forms such as tactical constructs do not necessarily appear as environment constants. Static help entries make them discoverable without pretending they have ordinary types.

Why it is designed this way

Leant has three deliberately separate text readers: the JSON protocol parser, the lightweight command classifier, and Lean itself. Reusing the classifier to interpret declarations or reusing formatted `##print` text as semantic type evidence would cross trust boundaries in the wrong direction.

Chapter 11

Interactive state and command execution

11.1 Reading `ReplState` by ownership domain

The record in `Main.hs` is large, but its fields form coherent groups.

Domain	Representative state and invariant
Backend lease	Optional process, immutable backend configuration, project directory, timeout. Environment IDs are meaningful only inside the current process.
Interactive Lean state	Current environment ID, base environment, snapshot base, imports, replayable history, undo stack, loaded-file identity and timestamps.
Derived environment state	Browse environment, synthesis base/environment, cached replay prefix. These may lag only when explicitly marked invalid and must be rebuilt before use.
Generated-value state	`it` counter, most recent `sorry`, private generated names, completion cache. Generated bindings have special invalidation rules.
Synthesis state	Engine choice, steps/budgets, classical/library switches, ratings, provider world and cache, synthesis-scoped generated splices.
Prove-mode state	Current proof session, goal information, splice counter, proof-local candidates. Leaving prove mode must remove or invalidate them.
Presentation state	Color mode, interactivity, timing, transcript capture, output sink, prompt/completion behavior.

The state record is mutable through Haskell updates, but many operations construct a complete replacement substate before installing it. The code is easiest to reason about when every field is assigned one of three roles: durable replay input, process-local handle, or derived cache.

11.2 Ordinary command execution

For a non-colon Lean command, the loop roughly performs:

1. classify and collect a complete input block;
2. ensure a live backend, replaying the session when required;
3. build a request payload containing the current environment ID and command;
4. run it under the configured timeout;
5. inspect backend errors, messages, and the new environment ID;
6. if successful, call the state-advance path to append history and push the previous state onto the undo stack;
7. invalidate derived browse/synthesis/provider data according to the declaration's semantic effect;

8. format and emit messages through the centralized output path.

The central `emit` function is more than convenience. Transcript capture, tests, color policy, and interactive output all observe the same sequence. Code that writes directly to stdout can make snapshots or golden tests appear nondeterministic.

11.3 Advancing environments

The state advance function updates the current Lean environment, replay history, undo stack, generated-value counter, and derived cache validity as one operation. Ordinary user declarations increment the provider-world generation and clear provider/completion caches. Generated `it` declarations are treated specially: they make the value available to subsequent commands but do not invalidate the provider world, because otherwise every synthesis result would evict the provider discovery that produced it.

This distinction is semantic, not based solely on a textual prefix. Generated definitions use a private mangled name convention and are recognized during replay and scanning.

11.4 Generated `it` bindings

Leant gives verified values convenient names `it`, `it1`, and so on, but it cannot safely declare those ordinary names repeatedly. Internally it uses private generated names carrying a monotonically increasing counter. A source scanner rewrites bare references to the friendly names outside strings and comments. Output is rewritten in the opposite direction.

The scanner must understand enough lexical structure to avoid changing:

- a substring inside a longer identifier;
- string and character literals;
- line comments and nested block comments;
- already mangled generated names;
- numeric suffixes that refer to older results.

Verified synthesis results are installed from worst to best. The newest generated definition is therefore the highest-ranked survivor and receives the bare `it` view. This order also makes undo intuitive: undoing once removes the best result first and reveals the next generated value if present.

A generated definition may require `noncomputable`. Leant first tries the ordinary declaration and retries with the modifier when the backend reports the relevant failure. Only successful declarations enter history.

11.5 Undo

The undo stack stores enough state to restore the prior environment and history boundary. Undo cannot cross the durable snapshot base because the older process-local environment IDs no longer exist and the snapshot deliberately defines a reconstruction root.

On undo, Leant restores:

- the previous environment ID;
- replay history length and contents;
- synthesis-derived state or invalidation markers;
- the generated `it` counter derived from surviving history;

- provider-world/cache state according to whether an ordinary declaration was removed.

The counter is not merely decremented blindly. Replay and undo scan recognized generated names so gaps or a loaded snapshot cannot cause name reuse.

11.6 Reset, reload, and load

These commands intentionally differ in reconstruction policy.

- `:reset` starts a replacement backend without replaying the old history. It preserves user preferences such as timeout, color, engine, and search settings while clearing declarations/import state to the baseline.
- `:reload` re-executes the currently loaded file against a fresh appropriate base, replacing its prior effects.
- `:load` starts without replaying the old session and establishes a new file/import/history root.

The "without replay" path is crucial. If history itself contains a command that now crashes the backend, a recovery command must not execute that command before it gets a chance to clear or replace the state.

Failure mode to keep in mind

A generic `ensureBackend` that always replays history before dispatch would make `:reset`, `:load`, and snapshot restoration unusable precisely when they are needed most. Replacement-process creation and old-session replay are separate operations in Leant.

Chapter 12

Transactional replay and snapshots

12.1 Why replay is transactional

After a timeout or server death, all old Lean environment IDs are invalid. Leant must create a replacement process and reproduce the session from durable inputs. A naive implementation might update `ReplState` after every replayed command. If command k fails, the interactive state would then point partly into the new process while history, undo, caches, and snapshot ownership still describe the old process.

Leant instead constructs a `ReconstructedSession` off to the side. Only after snapshot/import restoration and every history command succeed does it install the new backend and environment IDs. On failure, the partial replacement process is terminated and the old logical state remains available for reset or diagnostic commands.

12.2 Reconstruction sources

A session has one of two principal roots:

1. a base Lean environment plus a replayable import sequence;
2. a validated snapshot artifact with companion metadata and optional tooling artifact.

From that root, Leant replays ordinary history commands in order. The derived browse and synthesis environments are invalidated or replayed independently because their payloads and purposes differ from the interactive environment.

12.3 Pure replay accumulation

`src/Leant/Session/Replay.hs` contains the pure accumulator. Given a starting environment and a sequence of successfully executed history records, it computes the resulting current environment, undo boundaries, surviving history, and generated `it` counter. If any command fails at the effectful layer, no partial accumulator is returned as installable state.

Generated names are recognized during replay. This preserves counters across process replacement and prevents a newly synthesized `it` from colliding with a replayed definition.

Undo entries cannot precede the reconstruction root. If a snapshot compacted a long history, the new undo stack begins at the snapshot base even though the source session had older commands.

12.4 Import reconstruction

A Lean backend may report success while ignoring or degrading a missing import in ways that are not obvious from one response field. The import reconstruction path therefore probes the resulting environment after imports and

records the actual base. This avoids installing a session whose history IDs are valid but whose declaration universe differs from the old one.

12.5 Snapshot structure

`src/Leant/Session/Snapshot.hs` manages a snapshot as a small artifact family:

- the upstream Lean environment artifact, normally an `.olean`;
- an optional tooling companion needed by the interactive backend;
- a versioned sidecar containing format metadata, sibling names, fingerprints, sizes, and synthesis-tooling ABI information;
- durable temporary copies owned by a `SnapshotBase` value while the session is live.

The sidecar's fingerprint is a streaming FNV-1a 64-bit value accompanied by byte count. It is explicitly a corruption/identity check, not a cryptographic signature. The tooling ABI fingerprint incorporates serializer and prelude source so a snapshot created by an incompatible synthesis bridge is rejected rather than used with subtly different fragment semantics.

12.5.1 Validation

Snapshot loading checks:

- sidecar format version;
- expected sibling filenames and presence;
- byte sizes and fingerprints;
- tooling ABI compatibility;
- whether optional companions are consistently declared and present.

Only a validated artifact family becomes a `SnapshotBase`. The temporary ownership type ensures files needed by the live session are not deleted while state still refers to them.

12.5.2 Publication

Publishing a snapshot uses temporary names, sibling placement, rename, backup, and restore. This protects an existing valid snapshot if publication of a new family fails halfway through. The implementation accounts for Windows rename semantics, where replacing an existing file is less uniform than on POSIX systems.

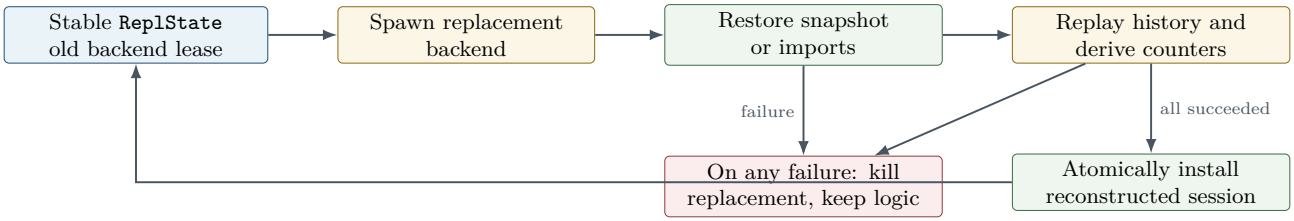

Figure 12.1: Transactional process reconstruction. The old logical state is never mutated into a half-replayed state.

12.6 Derived synthesis replay

src/Leant/Synth/Replay.hs contains a pure prefix comparison returning one of three decisions:

- `Reuse`: cached derived synthesis state already corresponds exactly to current history;
- `ReplaySuffix`: cached history is a prefix, so only the new suffix must be executed in the synthesis environment;
- `ReplayAll`: histories diverge or the cache is invalid, so rebuild from the synthesis base.

This small module prevents the large main loop from embedding subtle prefix logic in several command paths. It also makes replay decisions independently testable.

12.7 Recovery scenarios

12.7.1 Timeout during an ordinary command

The request path kills the backend and marks it absent. No new environment ID or history item is installed. The next command that needs Lean starts a replacement and replays the prior committed session. The timed-out command is not automatically retried, because its completion status is unknown.

12.7.2 Server death during synthesis verification

The candidate is not accepted. The backend lease is dropped. Djex state can remain in memory, but every Lean environment identifier and provider serialization tied to the dead process is treated as invalid until reconstruction. The next user command triggers replay.

12.7.3 Unreplayable history

Normal commands report replay failure without installing the partial replacement. The user can still issue `:reset`, `:load`, or snapshot restoration because those commands create a replacement without replaying the bad history first.

Chapter summary

Leant treats process death as loss of process-local identity, not loss of the logical session description. History, imports, snapshots, and settings are durable inputs; environment identifiers, handles, and derived caches are leases that can be reconstructed.

Chapter 13

From Lean expressions to the synthesis fragment

13.1 Why the bridge needs its own language

Lean's elaborated expression language is far more expressive than Djinn's logic or Exference's type language. It contains dependent products, universe levels, metavariables, reducibility, projections, recursors, quotient machinery, coercions, type-class synthesis artifacts, and arbitrary constants. A direct recursive translation would either reject nearly every practical goal or silently erase dependencies.

Leant instead defines a fragment that serves three purposes at once:

1. a **search projection**: the structural type problem sent to Djex;
2. a **rendering guide**: exact binder, constructor, application, and provider metadata needed to reconstruct Lean source;
3. a **completeness ledger**: explicit reasons why a no-candidate result may or may not support a negative conclusion.

The implementation is in `src/Leant/Synth/Fragment.hs`. The module contains both the source of the Lean serializer payload and the Haskell parser/analysis code, keeping producer and consumer close enough to review together.

13.2 The serializer runs after elaboration

The payload uses Lean metaprogramming to inspect the elaborated target and relevant declarations. It is embedded as source text and sent through the backend. Running after elaboration gives it exact binder info, fully resolved constant names, instantiated universes, inductive metadata, and local context. It does not need to reverse-engineer surface notation such as `A x B` versus `Prod A B`.

The payload emits a small S-expression protocol rather than JSON. The format is optimized for recursive tagged trees and can be parsed with bounded, location-aware routines in Haskell. Strings and exact names are quoted or escaped by the producer; the consumer does not split on whitespace heuristically.

The serializer carries a depth budget. When the budget is reached, it emits an explicit `FDepth`-like marker. It does not replace deep structure with a plausible atom. This distinction is what later prevents a depth-limited projection from supporting refutation.

13.3 The `Frag` grammar

The principal constructors can be grouped as follows.

Fragment form	Meaning
`FArr a b`	Non-dependent function space selected for structural synthesis.
`FProd xs`, `FSum xs`	Product and sum structure, including binary and generalized views used by constructor translation.
`FTop`, `FBot`	Canonical inhabited and empty proposition-like forms.
`FAll visibility name body`	Sort-level universal binder; visibility records whether Lean expects an explicit or implicit source binder/application behavior.
`FInst display body`	Instance-implicit binder retained as a distinct slot. Legacy or unsupported forms can be represented but poison negative evidence.
`FExactContext class args body`	Exact class/context binder with structured argument metadata instead of a flattened atom.
`FVar key`	Bound structural variable whose identity is tracked through the serializer.
`FAtom safe key`	Opaque atomic source type. The safety bit says whether treating it as an uninterpreted atom is conservative for refutation.
`FApp safe key head args`	Retained nominal or higher-kinded application with exact head metadata and projected arguments.
`FParamInd`, `FInd`	Nonrecursive inductive families, with constructor and parameter metadata needed for structural translation and rendering.
`FParamRec`, `FRec`	Recursive families plus an explicit completeness classification and constructor schemas.
`FDepth`	The serializer stopped because of the projection bound; positive transport may still be possible, negative evidence is incomplete.

The exact constructor fields are richer than the summary above. In particular, inductive nodes carry canonical keys, displayed names, parameter counts, constructors, recursive occurrence information, and flags controlling which structural view is safe.

13.3.1 Safe and unsafe atoms

An atom built only from universally bound variables can often be treated as an uninterpreted proposition without making an impossible goal easier. A constant application whose hidden definition might expose constructors or implications cannot safely support the same negative argument. The serializer therefore labels atoms with a safety bit derived from source structure.

Positive synthesis is less demanding. A generated term can forward an unsafe atom as an opaque value and Lean will verify it. The bit becomes decisive only when Leant considers promoting a Djinn miss to refutation.

13.3.2 Exact contexts

Lean instance arguments are dependent function binders with special elaboration behavior. Flattening them to ordinary arrows would make generated terms mention dictionaries as explicit term arguments and would lose class-head correlations. `FExactContext` retains the class name and structured arguments. Provider discovery and rendering can then distinguish a constraint required to use a declaration from a normal premise available to arbitrary search.

13.3.3 Application heads

`FApp` retains a nominal head rather than pretending every application is structurally transparent. The head metadata separates proper-kinded direct applications from higher-kinded or partially applied heads. This feeds two later paths:

- the Djex translation can preserve exact higher-kinded provider assignments;
- the renderer can decide whether an argument is a genuine first-order fragment substitution or must remain source metadata.

13.4 Binder spines and slots

The renderer needs to align a generated lambda spine with Lean's original products. `Fragment.hs` therefore exposes a `Slot` view for leading arrows, explicit/implicit universal binders, and instance binders. A slot is not merely a type: it records source visibility and whether a generated term should contain a binder, a placeholder, or no textual argument.

For example, these goals can share a similar structural core but require different source text:

```
(a : Sort u) -> a -> a
{a : Sort u} -> a -> a
[a : SomeClass] -> ...
```

The first has an explicit type-level binder in surface syntax, the second an implicit one, and the third an instance slot. The engine may erase some of this distinction, but the fragment cannot.

13.5 Parsed goals and provider fragments

A `ParsedGoal` contains the target fragment, sort/universe classification, provider query, recursive and unsafe metadata, and any pre-serialized library premises. A `ProviderFrag` contains the exact Lean name, a fragment for the provider scheme, source binder names, and one or more correlated assignment vectors.

Provider arguments have three important forms:

- a proper fragment with a known kind arity;
- an exact fragment carrying tags for forall domains that must remain correlated;
- a nominal head with already supplied proper arguments and a remaining higher-kinded arity.

Kind arity and exact-domain vectors are bounded at the parser boundary. A malformed backend response cannot allocate an unbounded assignment tree in Haskell.

13.6 Analysis functions form the completeness ledger

The module contains many small recursive analyses. They should be read as proof obligations, not miscellany:

- `fragHasDepth` detects bounded serializer truncation;
- `fragUnsafeAtoms` collects opaque regions unsafe for negative evidence;
- `fragHasInstanceBinder` and `fragHasUnsupportedInstanceBinder` distinguish supported exact contexts from legacy/opaque ones;
- `fragProviderMayOpen` asks whether provider-oriented forall elimination could expose useful structure;
- `fragRecKeys` identifies recursive families relevant to constructor premise selection;
- `fragSpine` and `leadingTypeArgs` drive fitting and provider occurrence alignment;
- `fragVisibleForallVisibilities` preserves explicit/implicit source behavior;
- `fragRefusal` explains why a fragment cannot be translated for a selected engine;
- `glivenkoSplit` and `propAtoms` characterize the optional classical fallback fragment;
- `stripRecCtors` creates a library-search variant less likely to be flooded by closed recursive constructor terms.

No single Boolean means "complete." The final decision combines serializer depth, atom safety, context support, recursive-family completeness, nominal-application completeness, exact assignment retention, and engine progress.

Invariant

Every lossy projection must leave a machine-readable scar. If a new `Frag` constructor can hide a proof-relevant elimination or constructor, its analysis must prevent a candidate-free Djinn result from being reported as a source-level refutation.

13.7 Recursive-family representation

Recursive inductives are difficult because unfolding them freely yields infinite types, while treating them only as opaque loses useful constructor synthesis and case analysis. Leant serializes a bounded family schema:

- a canonical family key and source name;
- parameters versus indices;
- constructor domains and result indices;
- which fields are recursive occurrences;
- whether all relevant constructors and argument fragments were serialized completely;
- a fallback nominal identity for forwarding.

The engine bridge may add conservative constructor premises outside the goal's leading arrow spine. For a recursive goal, these premises enable one-layer construction or elimination without pretending the entire fixed point has been unfolded. Foreign recursive families are excluded from a library premise when their schema is not tied to the current goal or provider query.

13.8 Classical transformation

The fragment module can identify a restricted proposition-shaped region suitable for a Glivenko-style classical fallback. It splits the goal and enumerates proposition atoms without interpreting arbitrary Lean constants. The transformed search is always budgeted and used only for positive candidates. Even if Djinn exhausts the transformed formula, Leant does not report a source-level classical refutation from that lane.

Source trail

Read `src/Leant/Synth/Fragment.hs` in three passes: first the exported data types, then the embedded Lean serializer and S-expression parser, then the analysis/transform functions. Mixing those passes makes it easy to confuse producer invariants with properties recomputed by Haskell.

Chapter 14

Library premises, provider discovery, and caching

14.1 Why search inputs are staged

A live Lean environment can contain tens of thousands of declarations. Sending all of them to a type-directed synthesizer would increase branching, introduce accidental constants, obscure structural proofs, and make results highly sensitive to imports. Leant separates three premise sources:

1. **structural premises**: constructors and exact local structure derived from the goal;
2. **curated library premises**: a small rated and relevance-filtered set chosen from a maintained catalog;
3. **live providers**: declarations discovered semantically from the current environment for this goal.

Each source receives its own search lane and diagnostics. A structural result should not be delayed behind provider enumeration; a provider-free refutation should not be emitted before provider lanes that may construct a term have been tried.

14.2 Curated library candidates

Library mode starts from configured names and ratings. Relevance filtering considers goal heads, argument/result structure, class roots, and recursive-family keys. The selected declarations are serialized in one Lean batch, which reduces process round trips and ensures all schemes come from one environment state.

A serialized library premise is rejected when it is unsupported, depth-truncated, unsafe in a way incompatible with the intended lane, tied to an unrelated recursive family, or otherwise incomplete. Ratings influence ordering and Exference penalties, but the exact serialized scheme remains the semantic authority.

Before the library lane, Leant can strip recursive constructor introductions from the goal-side structural premise block. Without this step, constructors such as empty lists or base cases can flood a bounded candidate window with closed terms, preventing a useful library application from reaching verification.

14.3 Provider queries

`src/Leant/Synth/ProviderCache.hs` defines a semantic `ProviderQuery`. It canonicalizes root heads, result heads, and relevant structure while removing generated `it` names. Two textual goals that differ only in private interactive names can therefore share provider discovery.

A provider query is intentionally coarser than the full goal. Discovery asks Lean for declarations whose schemes can participate in the relevant head/argument relation; exact fitting happens later through serialized provider fragments and assignment validation.

14.4 Provider-world generations

The provider cache key includes a monotonically increasing `ProviderWorld`. Ordinary imports and declarations increment the world and clear caches because new providers or instances may have appeared. Generated `it` definitions do not increment it; they are transient results, not a reason to rediscover the entire environment.

The cache is a bounded recency list rather than an unbounded map. Hits move entries toward the front. The key is the pair of world and semantic query, so stale providers from an older declaration environment cannot be reused accidentally even when the textual goal is unchanged.

14.5 Discovery and exact serialization

A provider-discovery payload runs in Lean. It filters declarations using elaborated type information, then serializes the surviving schemes into `ProviderFrag` values. The result records exact names, binder order, implicitness, kinds, contexts, and correlated instantiation alternatives.

The correlation is crucial. For a provider morally equivalent to

$$\forall F\, a\, b.\ (a \to b) \to F\, a \to F\, b,$$

the two occurrences of F, and the domain/result occurrences of a and b, must receive one assignment vector. Generating independent per-occurrence guesses would admit impossible mixed instantiations.

Recent code in both repositories pays particular attention to:

- provider-local instance-head evidence;
- occurrence-local metadata fitting;
- exact implicit arguments that become visible in a result;
- contextual assignments shared across several provider arguments;
- higher-kinded heads with partially supplied proper arguments;
- five- and six-binder rank-N instantiation frontiers.

These are all manifestations of one invariant: an assignment is a vector with shared provenance, not a bag of independently plausible types.

14.6 Staged prefixes

Provider discovery can return many candidates. Djinn lanes use geometrically widening prefixes, typically small, medium, larger, and full. Early prefixes keep the proof environment focused and can produce a compact result quickly. Later prefixes recover completeness relative to the discovered provider set.

Exference is naturally ranking-oriented and can often consume the full prepared provider set in one bounded queue search, though Leant still applies ratings and exact capacity limits. Combined mode coordinates these policies rather than forcing identical prefix behavior.

A sound provider-free Djinn refutation is retained as a fallback, not immediately shown. Leant first gives constructive provider and library lanes a chance. If none yields a verified candidate, the earlier refutation may be reported, provided every source-level completeness condition still holds.

Why it is designed this way

The provider-free proof environment answers a strong structural question. The provider lanes answer the practical question users usually intend: “Can the current Lean environment build this?” Retaining the former while exploring the latter avoids both premature impossibility claims and loss of useful negative evidence.

14.7 Cache invalidation examples

Operation	New world?	Reason
Import a module	Yes	New constants, instances, notation elaboration targets, and providers can appear.
Declare an ordinary `def`/`theorem`/`instance`	Yes	It may itself be a provider or alter type-class closure.
Install verified generated `it`	No	It is a transient synthesis result; rediscovery would create self-feeding cache churn.
Undo an ordinary declaration	Effectively yes/invalidate	The visible provider universe shrinks and old entries may name unavailable constants.
Restart and replay identical history	Preserve semantic generation after reconstruction	Process IDs change, but the logical provider world is reproduced; process-local serialized caches may still be refreshed.
Reset/load a different session	Yes/new root	The declaration universe is unrelated.

Chapter 15

Engine orchestration and evidence policy

15.1 The synthesis outcome type

src/Leant/Synth/Engine.hs exposes an outcome that conceptually has three forms:

- candidate groups plus notes;
- a sound refutation, with a flag describing whether self-reference diagnostics were involved;
- no term under the attempted policies, plus notes explaining incompleteness, budgets, or omissions.

A candidate group is a list of Lean source spellings for one semantic neutral candidate. Grouping occurs before final verification because ambiguity may be purely syntactic: explicit constructor qualification, inserted type placeholders, or eta fitting can yield several plausible texts for the same generated term.

The module sets bounded constants for backend candidate windows, combined verification frontiers, attempted groups, and shown successes. At this snapshot, the visible result count is small, while internal windows are larger so rejected spellings do not starve later semantic candidates. The constants are named and centralized; code should not introduce ad hoc `take 5` operations that lose progress accounting.

15.2 Top-level phase order

The driver used by `:synth` follows this policy:

1. translate the goal and establish the fragment completeness ledger;
2. run the selected engine or engines with structural premises only;
3. if enabled and useful, run a separate curated-library phase;
4. discover providers, then run one or more widening provider lanes;
5. merge and deduplicate positive candidates in a stable source-aware order;
6. optionally run the bounded classical candidate lane;
7. retain or discard any Djinn negative evidence according to all completeness conditions;
8. return bounded candidate groups and notes to the verifier.

The exact path can short-circuit after enough candidates. It cannot short-circuit to a refutation before later constructive lanes required by policy have run.

15.3 Fragment-to-Djex translation

The engine bridge maps `Frag` into Djex neutral types, declarations, constructor schemas, contexts, provider assignments, and renderer maps. It introduces synthetic premises in a controlled binder order:

- conservative recursive constructor premises associated with relevant goal families;
- optional library or classical premises;
- the retained original goal binder spine.

The renderer later knows the exact sizes of the synthetic premise blocks and removes their lambda binders from the neutral candidate, replacing occurrences with actual Lean globals. Premise ordering is therefore an ABI between engine translation and rendering.

Invariant

A synthetic premise may make search convenient, but it may not leak into the displayed term as an extra user argument. Every introduced premise has a stable identity, exact source replacement, and a renderer-side stripping rule.

15.4 Djinn lane

The Djinn runner constructs a Djex environment over a standard session and submits a target such as `leantSynth`. Options control alternatives, candidate cutoff, and optional choice-point budget. Kinded provider assignments are supplied directly rather than reconstructed from rendered text.

The result is interpreted as follows:

- checked candidates are rendered to bounded Lean-variant groups and sorted with a compactness preference over the observed engine prefix;
- a Djinn uninhabitability result becomes Leant refutation only when no search budget was used, the Djex formula translation was complete, the `Frag` projection is complete for negative evidence, and provider/library policy permits the conclusion;
- a self-reference-only result becomes a diagnostic no-term outcome unless the command explicitly presents that distinction;
- an incomplete family, depth marker, unsafe atom, unsupported context, omitted plan, or truncated search turns a candidate-free result into “no term found,” not refutation.

This lane is the only source of negative evidence in ordinary synthesis.

15.5 Exference lane

The Exference runner maps textual Lean/Djex variable identities to the integer identifiers expected by the backend, constructs a relevance-biased session, and supplies exact providers. It runs a strict lane with unused binders forbidden. If that yields no candidate, it can run a relaxed lane that permits unused arguments.

Queue and step limits are explicit. Candidate groups are deduplicated stably before the result window. Backend metrics and projection notes remain available for diagnostics. Empty results, even with an empty queue, are always interpreted as no term found under policy.

The strict/relaxed split is useful for goals with logically irrelevant arguments. A strict lane tends to produce user-expected implementations; the relaxed lane preserves completeness of practical search for constant functions and ignored proofs.

15.6 Combined mode

Combined mode does not concatenate two lists. It constructs a fair frontier with backend identity preserved. A representative policy reserves an early prefix for Djinn's compact structural terms, exposes a broad Exference frontier for ranked provider/library terms, then lets Djinn fill additional slots and alternates tails. Stable deduplication occurs within each source and again across the merge.

Outcome merging follows asymmetric evidence:

- candidates from either engine are positive proposals;
- a Djinn refutation can survive only if the other lane has no verified candidate and all completeness conditions hold;
- an Exference no-term result never creates or strengthens a refutation;
- notes from both sides are combined without turning a heuristic diagnostic into logical evidence.

15.7 Bounded forcing and Haskell laziness

Both backend result streams may be lazy. Leant forces only the bounded prefix required for candidate grouping, plus progress sentinels where needed, under the command timeout. The forcing function evaluates enough structure that exceptions and runaway search occur inside the timeout boundary rather than later during printing.

This is a subtle operational boundary. Returning a lazy list from a timed block and forcing it afterward would make the timeout illusory. Conversely, forcing an entire stream would destroy incremental search and could diverge after already finding useful candidates.

15.8 Classical fallback

When enabled and the fragment passes a precise propositional-shape test, Leant applies a Glivenko/negation-style transformation and invokes bounded Djinn search. Only successfully reconstructed candidates are used. The lane never contributes negative evidence because:

- the transformation covers a restricted source fragment;
- search is budgeted;
- classical proof reconstruction introduces additional source assumptions or combinators whose availability must be checked;
- a miss says nothing about the original intuitionistic or classical goal outside that bounded encoding.

15.9 Decision table for candidate-free results

Condition	Outcome	Reason
Djinn exhausted; complete formula and fragment; no budget; required provider lanes exhausted productively	Refuted	All known lossy boundaries are absent and proof search established uninhabitability of the translated problem.
Djinn exhausted but `FDepth` present	No term found	Hidden deep structure might contain a constructor or eliminator.
Djinn exhausted but unsafe atom present	No term found	Opaque source constant may reduce to proof-relevant structure.

Condition	Outcome	Reason
Djinn choice-point/candidate budget reached	No term found, truncated note	Search did not establish exhaustion.
Djinn formula-plan family incomplete	No term found	A different positive-forall/recursive opening plan may admit a proof.
Only target reintroduction yields a proof	Self-reference diagnostic	Ordinary inhabitation remains unresolved or impossible without recursion.
Exference queue empty	No term found	Heuristic expansion and pruning are not a decision procedure.
Classical lane empty	No term found	Bounded constructive use only; no negative interpretation.
Any lane produced a Lean-verified candidate	Candidates	Positive evidence dominates prior fallback refutation.

Source trail

The engine chapter's source trail is `src/Main.hs` at the `:synth`/`:suggest` command path, then `src/Leant/Synth/Engine.hs`. Read the fragment-to-Djex translator before the backend runners, and read outcome merging only after understanding the distinct Djinn and Exference evidence contracts.

Chapter 16

Rendering neutral candidates back to Lean

16.1 Rendering is typed reconstruction, not pretty printing

src/Leant/Synth/Render.hs is nearly a second compiler. Its input is a checked shared generated AST plus the exact `Frag`, constructor maps, provider maps, type maps, premise counts, source binder metadata, and assignment alternatives. Its output is a bounded set of Lean source texts that are plausible elaborations of the same semantic candidate.

The renderer cannot simply replace Haskell constructors with Lean constructors. The two source languages differ in implicit arguments, anonymous constructor notation, pattern restrictions, dependent binder visibility, nominal qualification, and higher-rank instantiation. In addition, the neutral candidate may contain synthetic premise lambdas that do not belong in user source.

16.2 Pipeline overview

A representative render pipeline is:

1. validate and decompose the generated function clause;
2. remove synthetic recursive/library/classical premise lambda blocks;
3. replace occurrences of those premise locals with their exact Lean globals;
4. normalize patterns and introduce internal matches for refutable binders;
5. convert unused safe binders to wildcards;
6. globally uniquify local names and reject unbound locals or holes;
7. identify provider occurrences and attach exact visible assignment vectors;
8. enumerate a bounded set of provider-domain and ambiguous-forall alternatives;
9. fit the term to the original Lean slot spine, with optional eta expansion;
10. render an idiomatic spelling and a more explicit constructor spelling;
11. deduplicate textual results while retaining semantic grouping.

Each step consumes richer structure than the final string. Reordering steps can be incorrect. For example, provider occurrence matching must happen before names are rendered away, while final binder wildcarding should happen after usage is known.

16.3 Premise stripping

Engine translation may turn a source provider or library declaration into a synthetic leading function argument. The neutral candidate then applies a local binder. The renderer knows the premise block size and mapping. It removes the corresponding lambdas and replaces each local occurrence with the exact qualified Lean name.

The removal is structural. It does not search textual output for a binder name, which could collide with a user local. If a candidate rearranges or partially applies premises, the generated AST still identifies the local occurrence precisely.

Recursive constructor premises follow the same discipline. The renderer restores constructor names and visible parameter behavior from `CtorInfo` rather than preserving the synthetic premise as an extra argument.

16.4 Provider occurrences and assignment fitting

A provider can occur several times with different instantiations. The renderer matches each occurrence against the provider map and the visible type-application vector retained by Djex. It then chooses source assignment metadata compatible with that occurrence.

Only a **proper provider argument**—a direct fragment of proper kind—is eligible for ordinary first-order fragment substitution. Higher-kinded or nominal partially applied arguments retain their exact evidence but are not masqueraded as simple fragments. This prevents the renderer from inserting a type expression with the wrong arity merely because its head name matches.

When metadata admits several bounded assignments, the renderer produces variants. Lean elaboration later decides. This is preferable to an arbitrary renderer choice, provided the variant bound and order are deterministic.

16.5 Pattern normalization

The shared generated AST can represent patterns that Lean does not permit directly in every binder position. The normalization pass handles several cases.

16.5.1 Tuple patterns

A boxed-pair constructor pattern is rendered as a Lean tuple pattern when that is the idiomatic and accepted form. Nested patterns preserve constructor arity and binder order.

16.5.2 As-patterns

An as-pattern such as conceptually `x@p` is transformed into a binder for the whole value plus an internal match that destructures it. This preserves both uses without relying on Haskell-specific syntax.

16.5.3 Refutable lambda or let patterns

Lean may reject a refutable pattern directly in a lambda binder. The renderer introduces a fresh variable and moves the pattern into a `match`. For example, a generated tuple-pattern lambda can become:

```
fun t => match t with
  | (x, y) => ...
```

The transformation must preserve the types of all introduced binders and the branch-local scope. Constructor patterns with impossible alternatives may become `nomatch` only when fragment metadata justifies emptiness.

16.6 Global binder uniquification

Generated terms can reuse pleasant short names in disjoint scopes. Lean source, provider substitution, and output rewriting become safer when every binder is globally unique within the candidate. The uniquifier traverses patterns and expressions, creates fresh source-safe names, and rewrites local references by internal identity.

This pass rejects any reference not present in the environment and any surviving hole. It never resolves a free textual name by guessing that it refers to a global.

16.7 Fitting to the Lean slot spine

The fitting algorithm compares the candidate's lambda/application structure with the exact leading `Slots` derived from `Frag`. It can:

- insert source binders for explicit universal slots;
- omit or mark implicit slots as appropriate;
- insert `_` placeholders for implicit engine instantiations that Lean must see explicitly at a provider occurrence;
- preserve instance-implicit elaboration rather than turning it into an ordinary parameter;
- eta-expand a function when the candidate is semantically usable but has fewer explicit lambdas than the source goal expects;
- retain a non-eta form in a separate cohort when Lean may elaborate it directly.

Ambiguous trailing forall sites can yield several variants. The implementation bounds the product of alternatives and orders common idiomatic forms first.

16.7.1 Core fitting cases

The recursive `fitCore`-style logic aligns introductions, known constructor results, eliminations, and case branches. It uses fragment domains to annotate or constrain branch binders when the generated AST alone is insufficient. A constructor's source name and parameter count come from exact metadata, not from parsing a rendered type.

16.8 Source spelling variants

The renderer typically emits an idiomatic spelling first and a more explicit spelling second. Examples of variation include:

- anonymous `.inl`/`.inr` versus qualified constructor names;
- tuple notation versus explicit product constructors;
- omitted versus visible type arguments;
- direct function value versus eta-expanded lambda;
- compact match syntax versus fully qualified constructors.

Variants remain in one semantic group. Final result ranking should compare groups, not let one candidate consume the entire window with ten spellings.

16.9 Final Lean verification

The verification path lives in `Main.hs`. Candidate groups are tried best first; variants inside each group are tried in renderer order. A Lean payload elaborates the candidate against the exact original goal. A candidate is accepted only when:

- the backend request succeeds;
- the response contains no Lean errors or fatal condition;
- elaboration produces a term of the target type;
- the generated declaration or proof does not contain an admitted `sorry`;
- any required noncomputable retry succeeds.

The verifier stops after a bounded number of successful groups and attempted groups. Failures can be retained for verbose diagnostics but are not shown as candidates.

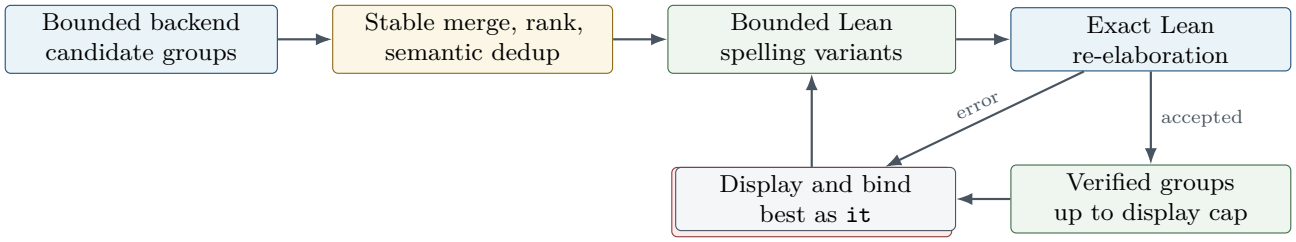

Figure 16.1: The candidate funnel. Internal engine checks and renderer fitting reduce defects, but the exact Lean target is the final gate.

16.10 Binding results

Outside prove mode, verified terms are declared in the current Lean environment. They are installed worst-first so the best group becomes the most recent generated value and therefore the friendly bare `it`. The new environment IDs and generated history records enter normal undo/replay machinery.

In prove mode, a candidate is not installed as an unrelated global theorem. Leant creates a proof splice applying the candidate to current hypotheses and exposes it as a proof-local `itN`. Leaving prove mode clears these splices so they cannot escape the goal context that justified them.

Invariant

Displayed candidates, generated `it` values, and prove-mode splices all come from the same verification gate. There is no "preview" path that bypasses Lean and later becomes executable state by accident.

16.11 Diagnosing a rejected candidate

A neutral candidate can be sound in Djex and still fail Lean for several reasons:

- the fragment erased a dependency that matters to elaboration;
- a provider's implicit argument needs a visible placeholder;
- a constructor requires qualification or a different parameter visibility;
- eta expansion is required to align a dependent binder;
- a pattern is refutable in a forbidden position;

- an instance argument cannot be synthesized in the current source context;
- a universe or sort relationship was transported opaquely but cannot be reconstructed;
- the candidate uses a synthetic premise in a way that cannot be replaced by the source declaration.

The correct fix may be a renderer variant, a richer fragment field, a stricter engine projection, or a provider metadata change. Do not weaken final verification.

Chapter 17

Prove mode and interactive proof synthesis

17.1 Entering prove mode

The `:prove` command establishes a Lean goal and a proof-session state. Subsequent tactic lines are sent to Lean with the current goal state. Responses update displayed goals and the proof environment. Ordinary command history and prove history are related but not identical: proof-local goals and generated splices cannot be replayed as global declarations without their context.

17.2 Tactic steps

A tactic step is authoritative only after the backend returns a new goal state. Leant displays messages, updates the prove counter, and records enough state for proof-local suggestions. A completed proof exits or offers to install the theorem according to command semantics.

17.3 Suggestions and synthesis

`:suggest` can ask Lean-oriented automation for tactics or terms. `:synth` in prove mode sends the current target through the same fragment/Djex/renderer/verifier pipeline described above. The difference is the source of local premises and the destination of accepted candidates.

The bridge serializes local hypotheses as exact arguments or contexts. A verified candidate may be a function expecting those hypotheses. Leant creates a splice that applies it in the current context, so the user can write `exact it` or inspect alternatives. The splice is verified in the proof environment before exposure.

17.4 Scope cleanup

Leaving prove mode clears proof-local `it` splices, cached goal translations, and counters tied to the old target. Provider cache entries keyed only by the semantic global world may survive, but any serialized local-provider metadata must not.

A useful invariant is: no value containing a local free variable may enter ordinary global history. The final Lean declaration path enforces this, but prove-state cleanup prevents even friendly-name rewriting from referring to stale locals.

17.5 Golden behavior

The integration corpus includes prove/suggest and synth/prove transcripts. These tests protect not only term correctness but prompt flow, goal display, splice naming, error recovery, and the boundary between proof-local and global generated values.

Chapter 18

Leant tests and maintenance seams

18.1 Unit tests

`test-unit/Spec.hs` is a broad unit suite covering JSON, classifiers, snapshot metadata and publication, replay decisions, fragment parsing and analysis, provider queries and cache behavior, Djex translations, assignment fitting, rendering, candidate grouping, and many regressions around rank-N providers and recursive families.

Because several production modules are pure, tests can construct exact `Frag`, generated AST, provider metadata, and history values without launching Lean. This is where boundary invariants should be tested exhaustively.

18.2 Golden integration tests

The `test` directory contains command scripts and expected transcripts. The runner starts Leant against the configured Lean backend and compares normalized output. Coverage includes:

- basic structural synthesis;
- both-engine frontier ordering;
- Djinn providers and refutation fallback;
- inductive and recursive families;
- instance-implicit contexts;
- curated library search;
- prove-mode synthesis and suggestions;
- provider argument rank-N behavior;
- constraint closure and quantified providers;
- contextual, structural, metadata, and higher-kinded assignments;
- visible results depending on exact implicit arguments;
- four-, five-, and six-binder rank-N frontiers;
- query-closed and recursive rank-N cases.

Golden tests are especially appropriate because source spelling, ranking, notes, and result binding are user-visible contracts. Pure tests should still establish the semantic reason for each golden line so formatting changes do not conceal a logic regression.

18.3 Natural refactoring seams

The current code already exposes several safe extraction boundaries:

1. Move command parsing and dispatch families out of `Main.hs` while keeping one explicit state-transition interface.
2. Define a typed algebra of backend requests above raw JSON payload construction.
3. Extract the top-level synthesis phase planner from `Main.hs`; keep `Fragment`, `Engine`, and `Render` pure or narrowly effectful.
4. Separate generated-value management into a module owning mangled names, scanner rewriting, declaration installation, replay recognition, and undo counters.
5. Preserve `ReconstructedSession` as the only installable result of replay, rather than exposing field-by-field mutation.

A refactor should not merge the interactive environment, browse environment, and synthesis environment simply because they all use Lean environment IDs. They have different payload histories and invalidation policies.

18.4 Where changes accumulate pressure

The largest change surfaces are:

- `Main.hs`: new commands or state fields can touch dispatch, help, replay, reset, transcript, and tests;
- `Fragment.hs`: a new semantic form touches producer source, parser, all recursive analyses, transformations, and completeness logic;
- `Engine.hs`: new provider evidence or Djex type structure touches both backend translations and outcome policy;
- `Render.hs`: new generated terms or fragment binders touch normalization, fitting, naming, variants, and verification tests.

The right response is not to avoid these files but to enter them through a checklist. Appendix C provides one.

Chapter summary

Leant's architecture is a sequence of explicit semantic boundaries wrapped in a large interactive orchestrator. The pure modules are already shaped like compiler passes. Maintaining correctness means preserving their metadata contracts and keeping process-local identities out of durable state.

Part IV

End-to-end traces and maintainer guide

Chapter 19

Concrete synthesis traces

The earlier chapters described each subsystem in isolation. This chapter follows representative requests across all boundaries. The examples are schematic Lean types; exact surface syntax and universe levels vary, but the source paths and state transitions are the ones to inspect in the pinned code.

19.1 Trace A: polymorphic identity

Consider:

```
forall {a : Sort u}, a -> a
```

19.1.1 Lean projection

The serializer emits an implicit universal binder, then an arrow from `a` to `a`. Both occurrences carry the same binder key. No depth, unsafe atom, provider, recursive family, or class context is present. The fragment is complete for structural negative evidence.

19.1.2 Djex translation

The bridge maps the binder to a rigid neutral type variable and the arrow to `FunctionType`. The original binder visibility is retained in the renderer's slot spine even though the engine type itself does not encode Lean's exact source notation.

19.1.3 Djinn

The positive forall is opened. LJT introduces the arrow antecedent and closes the atomic goal from the local assumption. The proof checker accepts the variable proof under one lambda. Lowering emits a generated lambda with one term binder; the type binder is represented by the surrounding polymorphic scheme rather than an ordinary term lambda.

19.1.4 Exference

The root goal opens the rigid forall scope, introduces one local value, and fills the result hole by selecting that local. Usage is perfect, so the strict lane accepts it. The independent checker reconstructs the variable at the same rigid type.

19.1.5 Rendering and Lean verification

The slot fitter sees one implicit type binder and one term arrow. An idiomatic variant is `fun x => x`; a forced explicit type-binder variant is normally unnecessary. Lean elaborates the candidate against the exact goal. Leant installs it as the newest generated `it`.

19.1.6 Where a regression appears

If the result is missing, likely boundaries are:

- binder keys were not preserved in the serializer/parser;
- a rigid variable was incorrectly converted to a flexible one;
- generated local scope was lost during simplification;
- slot fitting inserted an explicit type lambda unsupported by the target syntax;
- verification used a stale Lean environment ID.

19.2 Trace B: product swap

Consider:

```
forall {a b : Sort u}, a x b -> b x a
```

Here `x` stands for the product notation in this ASCII document.

19.2.1 Projection and neutral type

The serializer recognizes the product head and emits a product fragment rather than an unsafe nominal application. The bridge maps it to a boxed tuple type or an equivalent declared product constructor view, preserving parameter order.

19.2.2 Djinn path

The formula is $(a \wedge b) \to (b \wedge a)$. Search introduces the input, applies conjunction-left to obtain its fields, applies conjunction-right in reverse order, and returns a checked proof. Lowering may choose a tuple-pattern representation.

19.2.3 Exference path

The result hole selects tuple construction, creating holes of types `b` and `a`. The input local can be deconstructed. The node builder creates a child lexical scope for pattern-bound fields, fills the holes, and records both uses. The checker validates the case/tuple structure independently.

19.2.4 Renderer normalization

If the generated term uses a tuple pattern directly in a lambda, the renderer can emit the idiomatic form when Lean accepts it or rewrite to a fresh binder plus `match`. Binder uniquification must preserve the field order established by the constructor metadata.

This example is an excellent cross-language regression test because a wrong constructor parameter count, a swapped deconstructor scheme, or an incorrect pattern-scope map produces a term that looks plausible but fails final elaboration.

19.3 Trace C: sum elimination

Consider:

```
forall {a b c : Sort u}, (a -> c) -> (b -> c) -> Sum a b -> c
```

The structural fragment contains two function premises and a binary sum. Djinn translates the sum to a labeled disjunction and uses disjunction-left, creating one branch with an `a` assumption and one with a `b` assumption. Each branch applies the corresponding function. Proof lowering emits a case expression with exact constructor descriptions.

Exference selects elimination of the sum input, creates branch scopes, and searches each result hole under its branch-local payload. Scope forest correctness matters: the `a` value must not be visible in the `b` branch. The expression checker reconstructs branch-local schemes and verifies equal result types.

Leant's renderer restores `Sum.inl`/`Sum.inr` or anonymous constructor notation, normalizes branch patterns, and verifies the match. A branch-scope leak is likely caught by the Exference checker; a wrong Lean constructor spelling is caught by final verification.

19.4 Trace D: a live map-like provider

Suppose the goal is:

```
forall {a b : Type u}, (a -> b) -> List a -> List b
```

and the current Lean environment exposes a suitable map provider.

19.4.1 Structural phase

Without recursive constructor premises, the engines cannot generally implement mapping. With one-layer list constructors, they may construct only bounded shapes or recurse unsafely; Leant's recursive policy does not invent general recursion. The structural phase therefore usually returns no general term and, because recursive-family projection may be incomplete for elimination, should not immediately refute.

19.4.2 Provider discovery

The provider query records a list-like result head, function relation, and relevant root structure. Lean discovery finds a declaration whose elaborated scheme is morally

$$\forall a\, b.\ (a \to b) \to \mathrm{List}\ a \to \mathrm{List}\ b.$$

The serialized provider fragment records exact constant name, binder visibilities, kinds, contexts, and the correlated assignment vector tying the goal's `a` and `b` to the provider's binders.

19.4.3 Djinn provider lane

Leant translates the provider to a synthetic premise plus kinded assignment evidence. The formula plan must retain the provider's argument relationship. Djinn can then prove the goal by forwarding the premise at the exact instantiation. The checked proof lowers to a generated global/premise application with visible type applications when necessary.

19.4.4 Exference provider lane

Exference includes the provider in its rated global environment. Selecting it for the `List b` result creates holes for the function and list arguments. Locals fill those holes. The occurrence-local assignment is checked and carried into the expression.

19.4.5 Renderer

Synthetic premise lambdas are stripped and replaced with the actual qualified Lean provider name. The renderer decides whether type arguments can be omitted, need `_` placeholders, or should be visibly supplied. Several variants may be generated. Lean picks the first that elaborates.

19.4.6 Cache behavior

Repeating the same semantic goal after only installing generated `it` values should hit the provider cache. Declaring a new map-like function or instance increments the provider world and forces rediscovery.

19.5 Trace E: rank-N forwarding versus elimination

Rank-N behavior is easiest to understand by separating two cases.

19.5.1 Opaque forwarding

A goal may merely pass a polymorphic value through unchanged. The nested forall should remain a `TypeAtom` or opaque formula symbol. Both engines can prove identity at that atom without opening it. This is why "all foralls open" is not the only Djinn plan.

The source renderer does not need to know the internals of the value; it binds and returns it. Final Lean verification confirms exact rank-N type equality.

19.5.2 Elimination at an exact assignment

A different goal consumes a polymorphic argument at a particular type. Ordinary first-order search cannot choose that instantiation soundly. Leant's serializer and provider metadata produce an exact assignment vector; Djex validates kind, closure, correlation, and source scheme; Djinn adds a controlled instantiation axiom or Exference inserts visible type application under a rigid-scope plan.

The generated AST retains the visible type application. Leant maps the neutral type argument back to exact Lean syntax and may insert explicit placeholders for intervening implicit binders. Final elaboration ensures the higher-rank application is legal in the source context.

19.5.3 Why assignments are vectors

For a provider with six correlated binders, independently choosing six locally compatible arguments can create a globally inconsistent scheme. The assignment object is validated as one vector and retained at one provider occurrence. The recent five- and six-binder tests exist to prevent accidental truncation, permutation, or Cartesian explosion in this path.

19.6 Trace F: recursive inductive identity

Consider a goal `Tree a -> Tree a` for a recursive family.

The best term is forwarding identity, which requires no unfolding. The serializer therefore retains exact nominal family identity even if it also emits a one-layer recursive schema. Djinn's plan family includes an opaque interpretation that proves forwarding. An implementation that always opens recursive structure could instead generate a large case/rebuild term or mark translation incomplete unnecessarily.

Exference can select the local input directly, with no constructor expansion. Rating and type complexity should prefer this node over a case split. The independent checker sees the same nominal family key on input and output.

The trace demonstrates the dual representation rule: structural exposure is optional search knowledge; opaque identity is mandatory transport knowledge.

19.7 Trace G: a sound no-term result

Consider a purely structural intuitionistic type with no providers, no unsafe atoms, no recursive incompleteness, no contexts outside the supported fragment, and no search budget. A candidate-free Djinn run can produce `ProvedUninhabitable`. Leant checks:

1. the serializer did not emit depth markers;
2. every atom is safe for the negative interpretation;
3. forall and recursive plan families are complete for the observed sites;
4. exact provider/premise policy is complete for the command mode;
5. the LJT search finished rather than truncating;
6. no candidate from library/provider/combined lanes verifies;
7. the result is not merely an Exference miss or a classical bounded miss.

Only then is the no-term result reported as refutation. This trace should be tested together with near-miss variants that add one `FDepth`, unsafe atom, or choice-point budget and therefore downgrade the verdict.

19.8 Trace H: backend timeout and replay

Suppose candidate verification sends a Lean payload that times out.

1. The request wrapper raises `RequestTimeout` and kills the backend.
2. The candidate group is not accepted and no generated definition enters history.
3. `ReplState` retains durable imports, snapshot base, history, settings, and Djex-side data, but drops the backend lease.
4. On the next Lean-requiring command, Leant starts a replacement process.
5. It restores the snapshot or imports, then replays committed history transactionally.
6. Only after all steps succeed are the new process and environment IDs installed.
7. Derived synthesis/provider data tied to process-local serialization is invalidated or rebuilt according to its replay policy.

If replay fails, the partial replacement is discarded. A reset/load command can start another backend without replaying the offending history.

Chapter 20

Debugging and operational reasoning

20.1 Classify the symptom before instrumenting

Most failures fall into one of five observable classes:

1. the goal is refused before search;
2. search returns no candidate;
3. search returns a candidate that an internal checker rejects;
4. the renderer produces no Lean spelling;
5. Lean rejects every spelling or process recovery fails.

These classes correspond to different modules. Start with the earliest wrong artifact rather than the final user message.

20.2 Goal refused before search

Inspect the serializer response and parsed `Frag`. Questions:

- Is the S-expression syntactically complete and within the depth bound?
- Did a dependent product become an unsupported atom or an exact context?
- Is a universe/sort binder represented with the expected visibility?
- Did a recursive family serialize all constructor domains?
- Does `fragRefusal` name a deliberate unsupported form or a parser mismatch?
- Is the synthesis environment current with respect to interactive history?

Add a regression at the `Fragment` parser/analysis level before changing engine code.

20.3 No candidate from Djinn

Capture:

- the neutral goal and synthetic premise order;
- formula plans and completeness flags;
- exact assignments retained per plan;

- search strategy, remaining choice-point budget, and candidate cutoff;
- LJT exhaustion versus truncation;
- self-reference diagnostic result.

A common cause is a plan that opens a forall needed opaque, or preserves opaque a site needed open. Another is assignment evidence rejected because alias expansion changed the occurrence path. Do not add an unrestricted premise to make the test pass; first establish the missing controlled translation.

20.4 No candidate from Exference

Capture:

- root goal after opening leading foralls;
- eligible locals/globals/providers and their exact schemes;
- queue capacity, steps, depth, identifier exhaustion, and prune counts;
- top-scoring nodes around the point the productive path disappears;
- strict versus relaxed unused-binder lane;
- class obligations and rigid-scope rejections;
- checker rejection counts for solved nodes.

If queue pruning is nonzero, an ordering change can alter completeness under the bound. A candidate may be generated but then rejected by constraint closure or the independent checker, so “solved node count” and “candidate count” should be observed separately.

20.5 Internal checker rejection

For Djinn, compare the formula and proof environment at search and check boundaries. The checker should not receive already-erased instantiation evidence. Verify binder freshness after normalization and constructor labels after formula compilation.

For Exference, compare the raw and simplified expressions, reconstructed expected type, residual constraints, rigid provenance, and global schemes. If raw passes but simplified fails, the simplifier likely crossed a binder or visible-type-application boundary. If both fail, the search expansion probably updated the expression and type state inconsistently.

20.6 Renderer produces no spelling

Inspect the generated AST before premise stripping and after each renderer pass:

1. Are synthetic premise counts and identities correct?
2. Are provider occurrences associated with assignment vectors?
3. Did pattern normalization introduce branch scopes correctly?
4. Did uniquification find an unbound local or surviving hole?
5. Does the candidate lambda spine fit the fragment slots?
6. Did bounded alternative enumeration drop every assignment?
7. Is a higher-kinded argument being incorrectly treated as a proper fragment?

Write a pure renderer unit test with an explicit generated AST and `Frag`. Do not require a full Lean process to reproduce a structural renderer defect.

20.7 Lean rejects every spelling

Retain the exact verification declaration and backend messages. Compare it with:

- the original goal text and elaborated target;
- binder explicitness and universe parameters;
- qualified global/constructor names;
- visible type arguments and placeholders;
- required instances;
- eta-expanded versus non-eta variants;
- pattern position restrictions;
- noncomputable requirements.

A rejection is valuable information: it identifies a mismatch between fragment/renderer assumptions and Lean. The right fix is to enrich or restrict that boundary, not to mark the candidate verified based on Djex alone.

20.8 Replay or snapshot failure

First distinguish transport death from reconstruction failure. Then inspect:

- backend configuration and Lake working directory;
- snapshot sidecar version, sibling names, sizes, and fingerprints;
- synthesis-tooling ABI mismatch;
- import probe results;
- the first replay history item that fails;
- generated `it` counter recognition;
- whether a reset/load path incorrectly attempted old replay;
- whether partial reconstructed fields were installed before failure.

The critical postcondition is binary: either a complete replacement session is installed, or the old logical state remains with no live lease. Any mixed state is a correctness bug.

20.9 A minimal diagnostic record

For a hard synthesis regression, capture this compact record in a test fixture or report:

1. repository SHAs and selected engine;
2. exact Lean goal and imports;
3. parsed fragment summary and negative-evidence ledger;
4. provider/library names and assignment-vector counts;
5. neutral Djex goal and premise order;

6. backend options and progress/evidence;
7. generated AST before rendering;
8. rendered variants and Lean error for each attempted variant.

This record is usually enough to reproduce the defect without an entire interactive transcript.

Appendix A

Module index

This appendix is a practical source map. It is intentionally selective: it names the files that own contracts or control flow, not every compatibility or test helper.

A.1 Djex shared layer

File	Read it for
synthesis/src/Language/Haskell/Synthesis/Name.hs	Checked identifiers, namespaces, definition names, and source-safe naming.
synthesis/src/Language/Haskell/Synthesis/Kind.hs	Kind vocabulary, proper/higher kinds, finite kind-tree checks.
synthesis/src/Language/Haskell/Synthesis/Type.hs	Neutral type AST, variable rigidity, spines, normalization, substitution, binders.
synthesis/src/Language/Haskell/Synthesis/TypeAtom.hs	Alpha-aware opaque nested polytypes and lexical keys.
synthesis/src/Language/Haskell/Synthesis/Constraint.hs	Structured class constraints and transformations.
synthesis/src/Language/Haskell/Synthesis/Declaration.hs	Shared declarations, constructors, classes, instances, schemes.
synthesis/src/Language/Haskell/Synthesis/Environment.hs	Opaque environment authority and strict indexes.
synthesis/src/Language/Haskell/Synthesis/Inventory.hs	Search-facing prepared inventory and recursive classification.
synthesis/src/Language/Haskell/Synthesis/KindInference.hs	Kind validation and provider assignment kind checking.
synthesis/src/Language/Haskell/Synthesis/Query.hs	Query request, contexts, options, cached query, provider evidence.
synthesis/src/Language/Haskell/Synthesis/Search.hs	Search batches, progress, completion, truncation reasons.
synthesis/src/Language/Haskell/Synthesis/Candidate.hs	Candidate, residual constraints, backend details, rendering boundary.
synthesis/src/Language/Haskell/Synthesis/Generated.hs	Common generated expression/pattern/clause AST, scope, simplification.
synthesis/src/Language/Haskell/Synthesis/Selection.hs	Bounded stream observation and candidate selection.
synthesis/src/Language/Haskell/Synthesis/Diagnostic.hs	Structured validation and search diagnostics.

A.2 Djinn

File	Read it for
djinn/src-core/Language/Haskell/Djex/Djinn.hs	Stable checked adapter and public session/query operations.
djinn/src-core/Language/Haskell/Djex/Djinn/Internal/Request.hs	Query preparation and provider evidence validation.
djinn/src-core/Djinn/Core.hs	Plan orchestration, premises, budgets, evidence, deduplication, ranking.
djinn/src-core/Djinn/Internal/TypeFormula.hs	Type views, formula compiler, forall/recursive plans, completeness.
djinn/src-core/Djinn/Internal/LJTFormula.hs	Formula symbols and proof-term language.
djinn/src-core/Djinn/Internal/LJT.hs	Sequent rules, lazy proof search, fairness, budgets, normalization.
djinn/src-core/Djinn/Internal/ProofCheck.hs	Independent proof reconstruction.
djinn/src-core/Djinn/Internal/ProofToGenerated.hs	Checked proof lowering to shared generated syntax.

A.3 Exference

File	Read it for
exference/src-core/Language/Haskell/Djex/Exference.hs	Stable checked adapter, projection diagnostics, metrics.
exference/src-core/Language/Haskell/Exference/Core/Types.hs	Exference-facing shared type views and substitutions.
exference/src-core/Language/Haskell/Exference/Core/Unify.hs	Tagged namespace unification, HK heads, opaque atoms, occurs checks.
exference/src-core/Language/Haskell/Exference/Core/Internal/ExferenceNode.hs	Complete search-node state and typed goals.
exference/src-core/Language/Haskell/Exference/Core/Internal/ExferenceNodeBuilder.hs	Scope-safe node mutation and fresh allocation.
exference/src-core/Language/Haskell/Exference/Core/Internal/Exference.hs	Queue scheduler, expansion, pruning, completion, lazy trace.
exference/src-core/Language/Haskell/Exference/Core/Expression.hs	Annotated generated expression and compatibility patterns.
exference/src-core/Language/Haskell/Exference/Core/ExpressionCheck.hs	Independent bidirectional reconstruction and constraint check.

A.4 Djex facade and frontends

File	Read it for
src/Language/Haskell/Djex.hs	Unified backend facade and capability matrix.
src/Language/Haskell/Djex/HaskellSrc.hs	Source declaration/type conversion and rendering bridge.
src/Language/Haskell/Djex/Package.hs	Package/tool discovery and package-level loading.
src/Language/Haskell/Djex/REPL.hs	Unified interactive orchestration.
src/Language/Haskell/Djex/REPL/Scope.hs	Visibility, qualification, ambiguity, imports/exports.
src/Language/Haskell/Djex/REPL/Workspace.hs	Module graph and workspace reload behavior.

File	Read it for
src/Language/Haskell/Djex/CLI.hs	Process modes and command-line entry.

A.5 Leant

File	Read it for
src/Main.hs	Interactive state, commands, ordinary Lean execution, synthesis phase entry, verification, generated values, prove mode.
src/Leant/Backend.hs	Child process, framing, timeout/death errors, discovery.
src/Leant/Json.hs	Protocol JSON parser and encoder.
src/Leant/Classify.hs	Multiline lexical classifier.
src/Leant/Session/Replay.hs	Transactional replay accumulator.
src/Leant/Session/Snapshot.hs	Snapshot family, sidecar, fingerprint, publication.
src/Leant/Synth/Fragment.hs	Lean serializer, parser, fragment grammar, provider and completeness metadata.
src/Leant/Synth/ProviderCache.hs	Semantic provider key and world-generation cache.
src/Leant/Synth/Replay.hs	Derived synthesis replay decision.
src/Leant/Synth/Engine.hs	Djex translation, search lanes, providers/library, merge, evidence.
src/Leant/Synth/Render.hs	Premise stripping, pattern normalization, fitting, variants, Lean text.

Appendix B

Important bounds and invariants

The following values describe the pinned revisions. They are part implementation limits, part denial-of-service boundaries, and part search-policy contracts. Before changing one, find every validator, serializer, plan enumerator, renderer alternative bound, and test that assumes it.

Bound	Value	Why it exists
Djex provider-instantiation candidates	32	Bounds exact evidence enumeration and validation.
Djex provider assignments	32	Bounds correlated alternatives per request/provider set.
Arguments in one provider assignment	6	Keeps exact vector validation and rank-N frontiers finite; covered by six-binder regressions.
Observed kind arity	64	Prevents unbounded higher-kinded head trees from source input.
Kind-tree node observation	129	Sentinel-style bound corresponding to a finite binary observation around the arity limit.
Leant exact forall-domain tags	128	Bounds serialized exact provider-domain metadata.
Djinn quintuple frontier cap	512 per orientation	Bounds open/opaque plan combinations while remaining complete through eleven sites for quintuple choices.
Exference queue capacity in Leant lane	1024	Keeps live provider/library search operational; exact pruning is reported.
Leant backend candidate window	60	Gives renderer/verification room after deduplication and rejection.
Leant combined verification frontier	24	Bounds cross-engine semantic groups entering expensive Lean verification.
Leant attempted result groups	12	Bounds verification work and user-visible latency.
Leant shown successful groups	5	Keeps interactive output focused while retaining alternatives.
Backend stderr drain	200 lines with short per-line timeout	Captures useful crash diagnostics without hanging after server death.

B.1 Invariant checklist

1. Every public authority has a validating constructor and hidden representation.

2. Every finite authority observes lazy input through a bound plus sentinel, never an unbounded `length` assumption.
3. Type atoms preserve alpha identity and free-variable hygiene while blocking structural decomposition.
4. Environment source order and all strict indexes advance transactionally.
5. Search evidence is represented separately from completion/truncation.
6. Djinn negative evidence requires complete translation and exhaustive unbudgeted search.
7. Exference never reports logical uninhabitability from heuristic exhaustion.
8. Every Djinn proof and Exference solved expression passes an independent internal checker.
9. Every Leant candidate passes exact Lean elaboration before display or installation.
10. Every lossy Lean-to-fragment projection records incompleteness explicitly.
11. Provider assignments are exact correlated vectors with kind and occurrence provenance.
12. Synthetic premises have stable identities and are removed structurally by the renderer.
13. Process-local Lean environment IDs never survive backend replacement without replay.
14. Session reconstruction is all-or-nothing.
15. Generated `it` bindings do not invalidate provider discovery, but ordinary declarations do.
16. Proof-local generated values never enter global history with free locals.

Appendix C

Change recipes

C.1 Adding a new shared type constructor

Suppose Djex must represent a new type form rather than preserving it as a `TypeAtom`.

1. Add the constructor and smart constructors in the shared type layer.
2. Define equality, ordering/key behavior, free variables, binder traversal, substitution, alpha normalization, finite-width validation, and source rendering.
3. Decide how it participates in function/application/constructor spines and kind inference.
4. Update environment and inventory traversal, synonym expansion, and recursive classification.
5. In Djinn, add a one-layer `TypeView` case and make polarity/opacity/completeness explicit. Update formula symbols, constructor descriptions, proof checking, and lowering only if structurally supported.
6. In Exference, update unification, zonking, occurs checks, rigid scope, node expansions, simplification, and the independent checker.
7. Update the generated AST only if terms need a new elimination/introduction form; do not add one merely because the type is new.
8. Update source frontends and Leant's fragment translation if the form crosses that bridge.
9. Add negative tests where the form is unsupported or opaque; add positive tests for forwarding and structural use separately.

C.2 Adding a new `Frag` constructor

This is one of the highest-risk changes because `Frag` is simultaneously search, render, and evidence data.

1. Extend the embedded Lean serializer and give the form an unambiguous S-expression tag and bounded fields.
2. Extend the Haskell parser with malformed, truncated, and width tests.
3. Define free variables, binder spines, recursion keys, unsafe atoms, provider openness, exact contexts, depth behavior, and source-display summaries.
4. Decide whether the form is structurally translated, transported opaquely, or refused per engine.
5. Add a completeness rule: can hiding this form invalidate refutation?
6. Update library/provider serializers and relevance filtering if declarations can contain the form.
7. Update fragment-to-Djex translation and exact assignment metadata.
8. Update renderer fitting, binder domains, constructor metadata, and source variants.

9. Add a pure round-trip fixture, engine tests, renderer tests, final Lean golden tests, and a negative-evidence near miss.

C.3 Adding a new generated expression form

1. Extend shared `Generated` validation, scope traversal, free locals, simplification, application spines, equality/deduplication, and render diagnostics.
2. Add production and checking to each backend separately. Search construction alone is insufficient.
3. In Exference, extend annotated expression conversion and bidirectional checking.
4. In Djinn, extend proof lowering only when a checked proof rule justifies the form.
5. Extend Leant's pattern normalization, uniquification, premise substitution, fitting, and both idiomatic/explicit renderers.
6. Verify every source target; a Haskell-renderable form may not have direct Lean syntax.

C.4 Adding provider metadata

1. Identify whether the data is semantic evidence, renderer evidence, ranking input, or diagnostic text. Use separate fields/types when it serves more than one role.
2. Serialize it from elaborated Lean declarations with exact occurrence identity.
3. Bound vectors and trees at the parser boundary.
4. Include it in semantic cache keys only if changing it can change provider eligibility or assignments.
5. Validate it against Djex schemes and kinds before search.
6. Retain enough provenance through formula plans or Exference nodes.
7. Match it occurrence-locally in the renderer; do not apply one provider-wide guess to every occurrence.
8. Add correlated-assignment tests with repeated variables and higher-kinded heads.

C.5 Changing a search bound

A bound change is a semantic change whenever progress or result order is user-visible.

1. Find the validating smart constructor and the runtime cutoff.
2. Check whether a sentinel observation relies on $N + 1$.
3. Check whether budgets are global, per plan, per lane, or per queue.
4. Update exact truncation reasons and tests.
5. Measure first-candidate latency, result ordering, checker rejection, and negative-evidence behavior.
6. Ensure Leant does not reinterpret a formerly truncated run as exhaustive after only changing display limits.

C.6 Adding a Leant REPL command

1. Add help and command parsing without colliding with Lean source classification.
2. Declare whether the command requires a live backend and whether old history must be replayed first.

3. Define state effects by ownership domain: durable history, process lease, derived cache, provider world, generated values, prove state, transcript.
4. Decide undo semantics and whether the command establishes a new reconstruction root.
5. Route all output through `emit`.
6. Test success, backend death, timeout, bad response, reset/replay interaction, and noninteractive script output.

C.7 Changing snapshot format

1. Increment the sidecar format version or define a backward-compatible parser explicitly.
2. Include every sibling artifact in name/size/fingerprint validation.
3. Update the synthesis-tooling ABI when serializer or required prelude semantics change.
4. Preserve old snapshot files until the new family has been fully written and validated.
5. Test publication failure at every rename step, especially on Windows.
6. Test that an incompatible snapshot is rejected before starting replay from it.

C.8 Extracting code from `Main.hs`

Use state-transition boundaries rather than arbitrary line ranges.

- A command module should receive an explicit capability/state view and return either a complete state replacement or a command result describing effects.
- Backend request construction should return typed payload/result values; raw JSON should remain in a narrow protocol layer.
- Generated-value management can own mangled names, lexical rewriting, installation order, replay recognition, and counters.
- Synthesis orchestration can own phase order and outcome merge while delegating exact Lean effects to callbacks.
- Transactional reconstruction should continue to return one installable `ReconstructedSession`; do not expose partial field setters.

Appendix D

Glossary

Authority type
An opaque type whose values certify that a validation procedure has succeeded.

Candidate group
Several Lean source spellings representing one semantic generated candidate.

Checked request
A backend-private request tied to a validated session and exact environment.

Completeness ledger
Metadata recording every projection, plan, or budget condition relevant to negative evidence.

Exact context
A class/instance binder represented with structured class head and arguments rather than an ordinary arrow.

Generated AST
Djex's common scoped expression and pattern language produced by both engines.

Kinded assignment
A correlated provider-instantiation vector retaining each argument's exact kind.

LJT
The contraction-free intuitionistic sequent-calculus family used by Djinn's proof search.

Opaque forwarding
Treating a complex type as one atom so a value can be passed unchanged without opening unsupported structure.

Prepared inventory
A checked search-facing projection of a Djex environment.

Provider
A live or curated declaration whose scheme may be applied to construct the current goal.

Provider world
Leant's generation number for the semantic universe of discoverable declarations and instances.

Residual constraint
A class obligation remaining after a candidate's expression has been reconstructed.

Rigid variable
A type variable that represents a scoped constant and cannot be solved by ordinary unification.

Slot
Leant's view of a leading Lean binder, retaining arrow/forall/instance and visibility behavior for fitting.

Type atom
An alpha-aware opaque nested type region that preserves variables but blocks structural decomposition.

Verification gate
Final Lean elaboration of a rendered candidate against the exact original target.

Appendix E

Pinned source entry points

E.1 Repository roots

- Djex pinned tree: `github.com/VladimirReshetnikov/Djex/tree/f50b5eac...`
- Leant pinned tree: `github.com/VladimirReshetnikov/Leant/tree/425131c...`
- Leant's submodule declaration: `.gitmodules`
- Leant's multi-package build: `cabal.project`

E.2 Fastest route for a code review

For a change in synthesis behavior, review in this order:

1. public/bridge type changes in Djex shared modules;
2. opaque smart constructors and their tests;
3. the selected backend adapter and independent checker;
4. Leant fragment serializer/parser and completeness analyses;
5. Leant engine translation and provider assignment construction;
6. Leant renderer variants and final Lean golden tests;
7. evidence/progress handling and regression cases for candidate-free outcomes.

For a change in interactive behavior, review:

1. command parsing and help;
2. backend requirement/replay policy;
3. complete `ReplState` transition;
4. undo/reset/load/snapshot behavior;
5. provider and derived-environment invalidation;
6. transcript and noninteractive golden output.

Conclusion

Djex and Leant are best understood not as one theorem prover and one shell, but as a chain of explicitly checked translations. Djex gives two very different engines a common semantic perimeter. Djinn contributes proof-producing structural search and carefully qualified negative evidence. Exference contributes ranked typed-hole search, richer source-oriented application behavior, and a separate reconstruction checker. Leant projects elaborated Lean goals into a bounded fragment, plans structural/library/provider lanes, preserves exact instantiation metadata, reconstructs source syntax, and asks Lean to decide the final claim.

The codebase's strongest recurring idea is that uncertainty should have a type or a flag rather than an optimistic comment. Opaque regions remain opaque. Truncation is recorded. Exact assignments carry provenance. Search proposals are checked. Process-local IDs are reconstructed transactionally. A candidate-free heuristic run remains a candidate-free heuristic run. These disciplines are what allow the system to grow toward more powerful synthesis without silently converting approximation into unsoundness.

For maintainers, the practical rule is equally simple: follow the representation ladder. At every boundary, ask what information was retained, what was intentionally hidden, what validator established the next authority, and which later checker closes the remaining gap. The answer usually points directly to the right module and the right regression test.